

队伍编号	MC25002885
题号	D

基于混合预测与约束规划的短途运输集成调度优化研究

摘要

本题聚焦于城市内部或同省范围内的短途运输优化问题，该环节作为物流网络的末端，其核心挑战在于应对需求随机波动、满足严格的发运时效、高效利用有限的车辆资源，并在引入新技术，如标准容器时做出决策。为解决上述挑战，本文构建了一套集成了高精度货量预测、优化需求生成与串点、考虑多目标与内生技术选择的集成调度模型，为短途运输提供一套数据驱动的、精细化的、且具备风险意识的运营决策支持体系。

对于问题一，鉴于短途运输货量呈现显著的时序依赖性，我们设计并实现了一种基于深度学习的 LSTM-MLP 混合预测模型。该模型利用长短期记忆网络（LSTM）捕捉历史包裹量的复杂时间动态，同时通过多层感知机（MLP）有效融合静态特征。为满足后续调度对精细时间信息的需求，模型首先预测未来一日的线路总包裹量，随后采用基于历史到达模式的比例分配方法，将日度预测量拆解至 10 分钟的时间粒度。最终为问题二、三提供了关键的、时间精细化的货量输入数据。

对于问题二，在获得 10 分钟粒度预测后，研究的核心转向运输需求的精确生成与初步调度。首先，通过模拟各线路在各自累积窗口内的货量累积过程，识别出满足车辆标称容量的满载运输任务及其最早可发运时间。窗口末尾的剩余货量则被识别为尾货任务。针对尾货运输效率低的问题，我们引入了基于集合覆盖的优化模型进行最优串点。利用 CP-SAT 求解器求解此模型，生成了最优化的串点任务列表。最后，将满载任务、串点任务及必须单独运输的尾货任务整合，基于简单的车辆可用性和初步周转时间估算，我们建立了约束规划模型对承运车辆进行了初步分配，形成了基础的运输需求清单与调度方案。CP-SAT 求解的最终方案的总成本为 56776 元，自有车平均周转率为 2.49。

对于问题三，研究的关键是将标准容器的使用决策内生到调度优化模型中。标准容器通过缩短装卸时间提升效率，但降低了容量。为精确权衡此利弊，我们对问题二的调度框架进行了显著扩展，继续采用约束规划（CP-SAT）技术构建了一个更为复杂的集成优化模型，并添加了相关约束。更重要的是，我们将优化目标函数进行了精细化调整，使其能够内生地评估容器决策对多重目标的影响。通过求解此扩展模型，我们得到了一套不仅确定了发运时间、车辆分配，而且为每个任务智能地决定了是否使用标准容器的、综合考虑成本、自有车周转率和车辆填充率的优化调度计划。CP-SAT 求解的最终方案的总成本为 47106 元，自有车平均周转率为 2.62，总成本下降了 17%，自有车周转率上升了 5%。

对于问题四，为评估问题三优化方案在现实应用的可靠性，本研究开展了全面的敏感性分析，旨在探究预测偏差对固定调度计划执行效果的影响。我们构建了涵盖总量偏差和时间漂移的多场景模拟框架。在每个模拟场景下，将问题三生成的固定调度计划应用于模拟的“实际”货量流，通过仿真计算实际装载量与滞留量，并评估由此产生的实际 KPIs。通过对比分析各场景 KPIs 与基准值的偏差，研究定量揭示了调度方案的鲁棒性特征：方案对总成本波动相对稳健，但对导致服务水平下降的预测偏差（总量偏高、时间延迟）敏感，运营效率指标也受负向偏差和时间延迟影响。其中，时间延迟被识别为最具破坏性的风险因素。

关键词：短途运输；车辆调度；约束规划；串点优化；多目标优化

目录

一、 问题重述	1
问题背景:	1
问题一:	1
问题二:	1
问题三:	1
问题四:	1
二、 问题分析	2
2.1 问题一的分析:	2
2.2 问题二的分析:	2
2.3 问题三的分析:	3
2.4 问题四的分析:	3
三、 模型假设与约定	4
四、 符号说明及名词定义	4
五、 问题一模型建立与求解	5
5.1 数据解读与预处理:	5
5.2 模型的建立:	5
5.3 算法实施:	7
5.4 问题一的求解:	9
5.5 问题一的结果分析:	12
六、 问题二模型建立与求解	12
6.1 数据分析及可视化:	12
6.2 模型的建立:	14
6.3 算法实施:	15
6.3.1 阶段一: 运输任务生成与最优串点	15
6.3.2 阶段二: 集成调度与车辆分配	16
6.4 问题二的求解:	18
6.5 问题二的结果分析:	19
七、 问题三模型建立与求解	20
7.1 模型的建立:	20
7.2 问题三的求解:	22
7.3 问题三的结果分析:	25
八、 问题四模型建立与求解	25
8.1 模型的建立:	25
8.2 算法实施:	26
8.3 问题四的求解:	26
8.4 问题四的结果分析:	30
九、 模型的评价	31
9.1 优点:	31
9.2 缺点:	31
十、 参考文献	31
十一、 附录	32
11.1 第一题主代码程序:	32
11.2 第二题主代码程序:	44
11.3 第三题主代码程序:	53
11.4 第四题主代码程序:	62

一、问题重述

问题背景：

短途运输是物流网络中的关键环节，由起始场地、目的场地和发运节点组成，负责将货物从末分拣点运送至营业部。这一过程对包裹的履约时效和客户体验至关重要，同时占用大量运力资源。为完成这一过程，需要对货量进行预测，得到运输需求，最重要的是完成运输车辆的调度，提高运输效率，降低运输成本。通过优化运输活动，可以达成这一运输目标。现有 5 个自有车队，针对运输需求和货量需要解决以下问题：

问题一：

建立一个货量预测模型，在每条线路的总货量细化到 10 分钟的颗粒度的情况下以及在给定的结果时间范围内预测未来一天各条线路的货量。需要特别关注线路编码为“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的预测结果。

问题二：

根据问题一的预测结果，确定各条线路包括发运车辆数、预计发运时间和串点方案的运输需求。目标是最大化自有车辆的周转率及所有车辆均包裹数，并最小化总成本。需要特别展示线路编码为“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的调度结果。

问题三：

假设使用一种可以减少自有车辆装车及卸车时间，但也会降低车辆的装载量的标准容器，且容器数量无限。在问题一的预测结果基础上，重新确定运输需求，完成与问题二相同的优化目标，并额外输出各运输需求是否使用该标准容器，需展示线路编码为“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的调度结果。

问题四：

评估问题一的货量预测结果出现偏差时，对问题三的调度模型优化结果的影响。

二、问题分析

2.1 问题一的分析：

问题一的核心目标是构建一个可应对短途运输业务中的资源规划与调度需求的货量预测模型。我们需要完成以下两项任务：未来单日总货量预测及 10 分钟粒度货量拆解。解决此问题的关键在于有效利用包含历史 10 分钟粒度实际包裹量、未来一天及近期的每日“预知”包裹量，以及线路基本信息的数据^[1]。其中，如何将“预知”包裹量有效融入模型是一个核心考量点。同时，历史数据中的时间序列特性以及不同线路可能存在的模式差异，也需要能够被模型捕捉^[2,3]。最后，如何将日度预测结果合理地拆解到 10 分钟粒度，使其符合实际业务中货量逐步累积的特点，是完成任务二的关键。

我们面临的挑战包括处理时间序列特性、融合外部预知信息、进行多粒度预测以及处理有限的历史数据特性。为了解决这些问题，我们首先通过特征工程，提取了时间特征和滞后特征；其次，我们通过数据融合技术，将实际包裹量数据与预知包裹量数据合并；接下来，我们设计了基于历史分布的拆分方法，以实现日度总货量到 10 分钟粒度的精细拆分；此外，我们针对数据跨度有限的问题，采用了聚合数据和默认值填充策略以确保稳健性；最后，我们结合长短期记忆网络和多层感知机的优点构建了一个 LSTM-MLP 融合模型，该模型能够同时捕捉时间序列的动态模式和目标日的静态信息^[4,5]。通过这些方法，可以解决构建精确货量预测模型的难点并构建模型。

2.2 问题二的分析：

问题二要求基于问题一输出的 10 分钟粒度货量预测结果，制定未来一天的详细运输与调度计划，需明确包括潜在的串点线路的各运输任务的发运车次数量、预计发运时间、具体的串点方案，并最终确定每个任务的承运车辆。难点在于组合优化与多目标决策的特性：如何将连续的 10 分钟货量预测，有效地转化为离散的、满足车辆容量和最晚发运节点约束的运输任务^[6,7]；对于货量不足整车的线路如何智能地组合以提高车辆装载率、减少车次，同时满足线路和容量限制，而简单的贪心策略可能导致次优解^[8,9]；如何在满足任务时间要求的前提下，将任务分配给具体车辆，特别是要高效利用有限的自有车辆资源，实现高周转率；

同时调度决策需同时考虑多个目标^[10]。为解决以上问题，可以设计并实施一个两阶段优化策略，首先实现运输任务生成与最优串点，此阶段的核心是将预测货量转化为明确的运输单元，并通过优化模型确定最佳的串点组合；其次实现集成调度与车辆分配，接收第一阶段生成的任务列表，利用先进的约束规划技术^[11, 12]，进行精细化的发运时间确定和车辆分配，同时优化多个运营目标。

2.3 问题三的分析：

问题三引入了标准容器，其核心特性在于为提高车辆的周转效率提供了可能性。然而，此效率提升伴随着单车有效载货容量的降低。因此，问题三的核心挑战在于建立一个能够内生决策的模型，即模型自身需要根据潜在的收益和成本进行权衡，智能地决定是否为每个运输任务配备标准容器，以达到全局最优的运营效果。此决策过程需要与问题二中的发运时间确定、串点组合、以及车辆分配紧密集成。同时，优化目标仍然是寻求自有车周转率最大化、所有车辆均包裹量最大化、以及总成本最小化这三个目标的最佳平衡点^[10]。

本文选择在约束规划和 CP-SAT 求解器的基础上进行扩展以应对问题三的挑战。问题三的关键创新与区别在于将容器使用从外部参数或后处理规则提升为模型内生的核心决策变量。这意味着模型不再仅仅基于固定的任务属性进行调度，而是同时决定任务的执行方式和执行计划，从而实现更深层次的全局优化。

2.4 问题四的分析：

问题四需要基于问题 3 评估当问题 1 的货量预测结果出现偏差时对调度模型优化结果的影响。由于任何预测模型均无法完全避免与实际情况的偏差^[3]。因此，评估该优化调度方案在面对预测不确定性时的表现，即其鲁棒性，对于评估方案的实际应用价值至关重要^[13]。

本文旨在通过系统的敏感性分析，深入探究问题一预测结果中不同类型与程度的偏差，对问题三优化调度计划在执行效果上的具体影响^[14]。研究的核心目标包括：定量评估预测偏差对关键绩效指标、确定调度方案对哪种类型的预测偏差以及何种偏差方向更为敏感、基于敏感性分析结果，对问题三提出的优化调度方案在应对预测不确定性方面的整体稳健性做出评价以及为提高预测与调度系统的整体韧性提供策略性建议。为实现上述目标，本研究采用基于多场景仿真的敏感性分析框架进行求解。

三、模型假设与约定

- (1) 假设所提供的历史货量数据、预知货量数据、线路信息、车队信息以及串点规则是准确、完整且能代表研究期间及未来短期运营特征的。
- (2) 假设在问题一中构建的货量预测模型能够有效捕捉历史数据的潜在规律，并为未来一天提供合理可靠的 10 分钟粒度货量预测基准。
- (3) 假设所有车队所使用的运输车辆均为题目所述的单一标准车型，具有固定的日固定成本、固定的装卸能力和固定的装卸时长。
- (4) 假设 10 分钟粒度的预测包裹量累积达到车辆容量阈值即可触发一个运输需求，同时所有生成的运输需求必须在对应的最晚发运节点之前被服务。
- (5) 假设串点组合仅限于同一始发场地、同一发运波次的线路，且目的地数量不超过 3 个。在计算串点任务的成本和在途时间时，采用了简化规则。
- (6) 在车辆调度模型中，计算自有车辆完成任务后的可用时间是基于固定的“发运时间 + 装车时间 + 在途时长*2 + 卸车时间”公式。
- (7) 假设外部承运商的运力是无限可用的，并且其成本可以通过一个固定的乘数相对于自有车辆的变动成本进行估算^[6]。
- (8) 假设问题二、三的主观权重(w_{cost} , $w_{turnover}$, $w_{fillrate}$)选取了相对合理和均衡的基准权重组合，并假设其真实有效。
- (9) 假设所有外部车辆均不使用标准容器。

四、符号说明及名词定义

符号	含义
x_i	输入特征
b_j	偏置
R	线路
D	预测日
$V_{TotalPred,D}(R)$	日度总包裹预测量
$Ratio_{0600}(R_{base})$	平均历史比例
$V(R_{base}, W_{0600,D})$	每日窗口内的货量
w_{cost}	成本控制导向
$w_{turnover}$	资源利用率导向
$w_{fillrate}$	运营效率/环保导向

五、问题一模型建立与求解

5.1 数据解读与预处理：

由于附件格式并未统一，数据信息较为分散且存在缺失值，需要对附件数据进行整合与特征提取。

首先对附件 2、附件 3 的日期、时间相关列进行标准化处理，统一转换为 `datetime` 或 `time` 对象，确保后续处理的一致性，同时对“线路编码”进行清洗，去除潜在的空格，保证其作为合并键的准确性；其次需要对历史数据进行聚合，为构建日度预测模型，按“线路编码”和“日期”对附件 2 数据进行分组，计算每日的“总实际包裹量”，得到一个包含 线路编码，日期，总实际包裹量 的基础数据集；再者进行数据融合，将聚合后的日度实际包裹量数据与附件 3 提供的每日“预知包裹量”数据，以及附件 1 提供的线路基础信息进行合并，优先保留所有实际发生量的记录，并对可能缺失的“预知包裹量”采用 0 值填充策略；接下来完成特征工程以增强模型的预测能力，我们构建了从“日期”中提取“星期几”的时间特征，以捕捉潜在的周内周期性，提取了针对“总实际包裹量”的滞后特征，创建其过去 1 天、2 天、3 天和 7 天的滞后值，这些特征能够使模型直接学习历史数据的自相关性，还提取了静态特征，保留了目标预测日的“预知包裹量”和“星期几”作为当天的静态输入特征；最后进行缺失值处理，移除了因创建滞后特征而在数据初期产生的含有缺失值的行，确保模型输入的完整性。

经过上述处理，我们为每个线路编码构建了一个包含历史信息、预知信息和时间特征的结构化数据集，为后续模型训练奠定了基础。

5.2 模型的建立：

考虑到问题的特性，单一模型可能难以同时捕捉复杂的时序依赖和目标日的特定静态信息。为同时解决两个问题，考虑将 LSTM 模型以及 MLP 模型进行融合。接下来对两种模型进行基准模型考量。

首先是多层感知机即 **MLP 模型**^[6]，作为一种通用的非线性函数逼近器，其结构简单，训练速度快，可以直接利用构建的滞后特征、预知包裹量和星期几等信息进行预测。其基本原理是对输入特征进行加权求和，并通过激活函数（如 ReLU）引入非线性：

$$h_j = f\left(\sum_i w_{ij}x_i + b_j\right)$$

其中 i 是输入特征的维度数， j 是一层中神经元的个数， x_i 是输入特征， w_{ij} 是权重， b_j 是偏置， $f(\cdot)$ 是激活函数。然而，标准 MLP 缺乏内在的记忆机制，难以有效捕捉时间序列中长距离的依赖关系。

接下来是长短期记忆网络即 **LSTM 模型**，LSTM 是循环神经网络的一种变体，专门设计用于处理序列数据中的长期依赖问题^[4]。其核心在于引入了“门控机制”即输入门、遗忘门以及输出门来控制信息在网络中的流动和保留。

$$\left\{ \begin{array}{ll} f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) & \textcircled{1} \\ i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) & \textcircled{2} \\ \tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) & \textcircled{3} \\ C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t & \textcircled{4} \\ o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) & \textcircled{5} \\ h_t = o_t \odot \tanh(C_t) & \textcircled{6} \end{array} \right.$$

公式①表示遗忘门，决定从细胞状态中丢弃哪些信息。公式②和公式③表示输入门，用于决定哪些新信息被存放在细胞状态中。公式④表示细胞状态更新。公式⑤和公式⑥表示输出门，用于决定输出什么值。

其中 σ 是 Sigmoid 函数， $\tanh$ 是双曲正切函数， W 和 b 分别是权重矩阵和偏置向量， $\odot$ 表示逐元素乘积， h_{t-1} 是上一时刻的隐藏状态， x_t 是当前时刻的输入， C_t 是当前细胞状态。LSTM 能够很好地学习历史序列模式，但直接融合预测日特定的静态特征则不够自然。

为了结合两者的优点，我们采用了一种基于 Keras Functional API 构建的 LSTM-MLP 混合模型^[1]。该模型为双输入结构，可接收两个独立的输入流。当输入为序列输入时，该输入流将经过一个或多个 LSTM 层处理，以捕捉时间依赖性；当输入为静态输入时，该输入流将会由一个或多个全连接层即 MLP 结构处理。两者输出后会被拼接实现特征融合，融合后的特征向量再通过一个或多个 MLP 结构

进行处理，最终输出一个单一预测值。该模型设计如下：

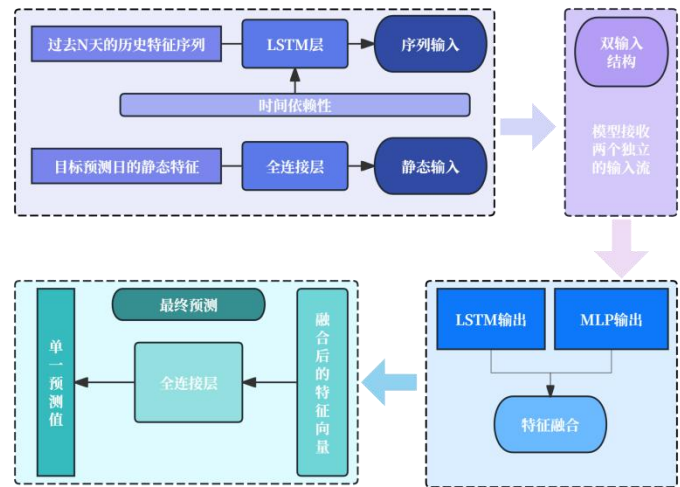


图 1 模型设计图

这种结构使得模型能够同时学习历史序列中的动态模式（LSTM）和目标日当下的关键静态信息（MLP），理论上能获得比单一模型更优的预测性能。我们为每个“线路编码”单独训练了一个这样的模型，以适应不同线路可能存在的独特模式。

5.3 算法实施：

首先运用以下算法对日度总量进行预测：

（1）数据缩放：在训练前对输入和输出进行归一化处理并缩放到[0, 1]区间，有助于模型稳定训练和加速收敛，同时为每个线路的每种输入和输出保存独立的缩放器以便后续反向转换。

（2）模型训练：使用准备好的历史数据对每个线路的融合模型进行训练。选用优化器 Adam 以及为均方误差的损失函数，并同时通过调整 epochs、batch_size 等超参数进行训练^[5]。

（3）预测执行：提取目标预测日前 N 天的实际历史特征作为序列输入，目标预测日当天的预知包裹量及星期作为静态输入；输入完成后，使用训练时的对应线路的缩放器对其进行转换；将缩放后序列和静态输入送入模型，得到预测结果；使用目标变量的缩放器对预测结果进行反向转换得到原始尺度的预测总包裹量，确保其不小于 0。

算法流程如下图所示：

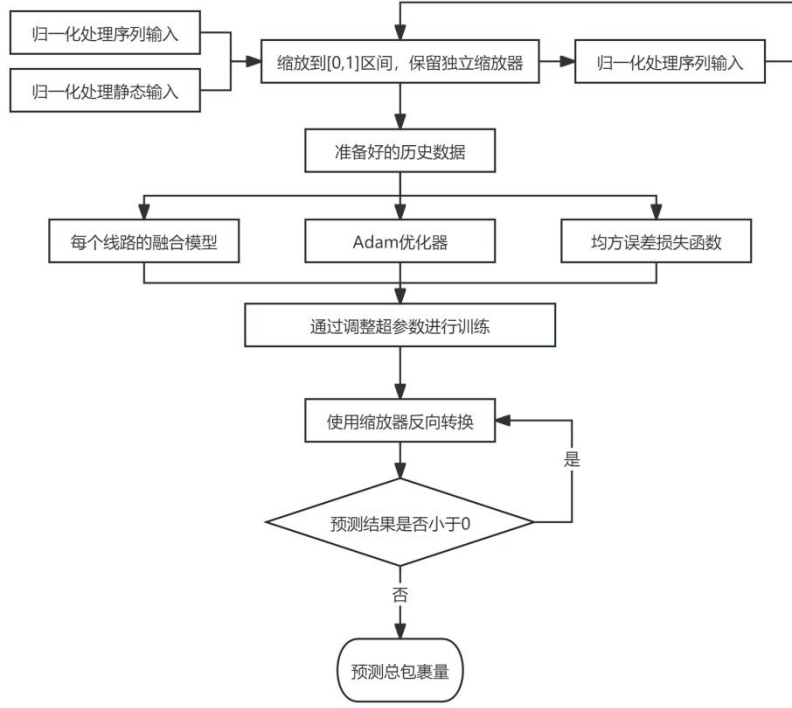


图 2 算法流程图

完成上述步骤后，可以得到每个线路编码在 2024 年 12 月 16 日的预测总包裹量，并将其整理至 结果表 1。

接下来将预测结果拆分至 10 分钟颗粒度。本文运用基于历史数据的节点波次归属比例估计方法对货量预测时间进行粒度细化。

虽然日度总包裹预测量 $V_{TotalPred,D}(R)$ 在宏观上反映了线路 R 在预测日 D 的货量，但无法直接指导运营。短途运输围绕特定的发运节点即 N_{0600} 和 N_{1400} 组织，且各有明确的货量累积时间窗口：

$$W_{0600,D} = [D - 1 \text{ 21:00}, D \text{ 06:00})$$

$$W_{1400,D} = [D \text{ 11:00}, D \text{ 14:00})$$

因此，在进行 10 分钟粒度拆分前，必须估算量 $V_{TotalPred,D}(R)$ 中应归属于 N_{0600} 波次的比例，以确保货量被分配到正确的时间区间，这是后续精确拆分和调度的前提。

我们基于附件二中历史数据的稳定性假设，设计了一种方法来估计基础线路 (R_{base}) 归属于 6 点波次的平均历史比例 $Ratio_{0600}(R_{base})$ 。通过分析历史数据，可以计算每日发生在 $W_{0600,D}$ 窗口内的货量 $V(R_{base}, W_{0600,D})$ 占当日两个关键窗口总

货量 $V(R_{base}, W_{0600,D}) + V(R_{base}, W_{1400,D})$ 的比例，然后对所有有效历史日求取平均值。具体公式如下：

$$Ratio_{0600}(R_{base}) = Average_{D \in D_{R_{base}}} \left(\frac{V(R_{base}, W_{0600,D})}{V(R_{base}, W_{0600,D}) + V(R_{base}, W_{1400,D})} \right)$$

其中 $D_{R_{base}}$ 为 R_{base} 有效计算比例的历史日期集合。计算时按基础线路聚合数据，忽略具体线路编码的后缀（-0600/-1400）。若某基础线路无足够历史数据计算比例，则采用默认值 0.5 作为回退策略。

估算得到的 $Ratio_{0600}(R_{base})$ 用于将预测日 D 的总预测量 $V_{TotalPred,D}(R)$ 拆分为归属于两个波次的量：

$$V_{Pred,0600,D}(R) = V_{TotalPred,D}(R) \times Ratio_{0600}(R_{base})$$

$$V_{Pred,1400,D}(R) = V_{TotalPred,D}(R) \times (1 - Ratio_{0600}(R_{base}))$$

这两个量随后将分别使用对应时间窗口 $W_{0600,D}$ 和 $W_{1400,D}$ 的历史到达模式 Profile 进行 10 分钟粒度的最终拆分，确保预测量在时间维度上的分配更符合业务逻辑。

5.4 问题一的求解：

利用模型，可以求得所有站点的预测结果如下图图 3 所示：

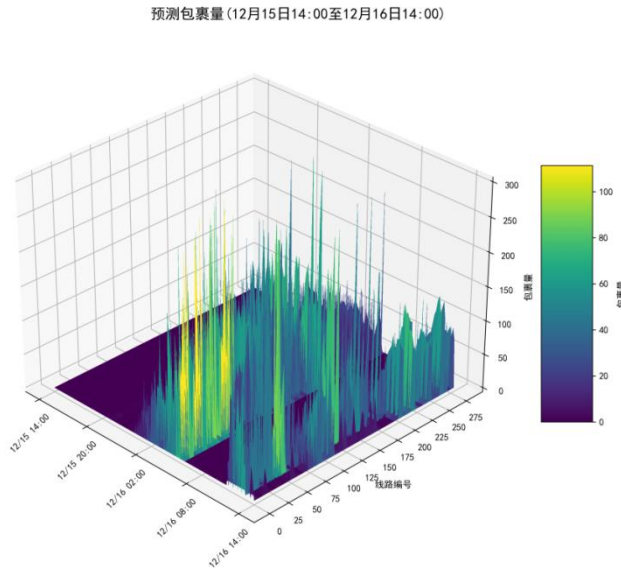


图 3 12 月 15 日 14:00 至 12 月 16 日 14:00 预测包裹量

同时针对线路“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的可视化如下图图 4 和图 5 所示，详细预测结果如下表表 1 和表 2 所示：

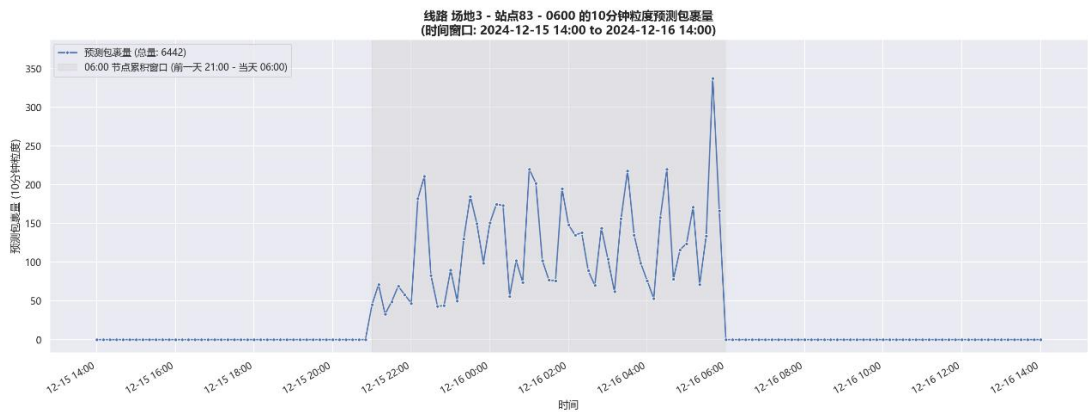


图 4 场地 3 - 站点 83 - 0600 预测结果

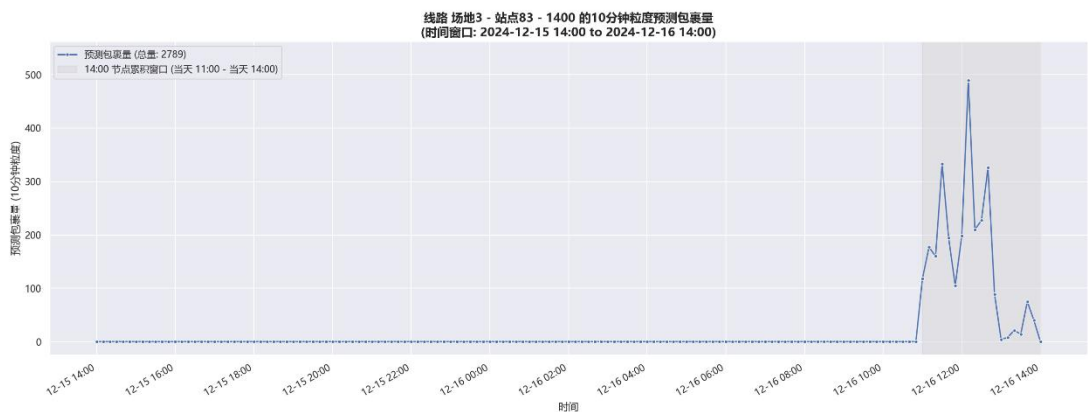


图 5 场地 3 - 站点 83 - 1400 预测结果

表 1 站点总货量预测结果

线路编码	日期	货量
场地 3 - 站点 83 - 0600	2024/12/16	6261
场地 3 - 站点 83 - 1400	2024/12/16	2746

表 2 站点 10 分钟颗粒度货量预测结果

线路编码	日期	分钟起始	包裹量	线路编码	日期	分钟起始	包裹量
场地 3 - 站点 83 - 0600	2024/12/15	21:00:00	44	场地 3 - 站点 83 - 0600	2024/12/16	03:00:00	101
场地 3 - 站点 83 - 0600	2024/12/15	21:10:00	69	场地 3 - 站点 83 - 0600	2024/12/16	03:10:00	60
场地 3 - 站点 83 - 0600	2024/12/15	21:20:00	32	场地 3 - 站点 83 - 0600	2024/12/16	03:20:00	151
场地 3 - 站点 83 - 0600	2024/12/15	21:30:00	48	场地 3 - 站点 83 - 0600	2024/12/16	03:30:00	212

场地 3 - 站 点 83 - 0600	2024/12/15	21:40:00	67	场地 3 - 站 点 83 - 0600	2024/12/16	03:40:00	131
场地 3 - 站 点 83 - 0600	2024/12/15	21:50:00	56	场地 3 - 站 点 83 - 0600	2024/12/16	03:50:00	96
场地 3 - 站 点 83 - 0600	2024/12/15	22:00:00	46	场地 3 - 站 点 83 - 0600	2024/12/16	04:00:00	73
场地 3 - 站 点 83 - 0600	2024/12/15	22:10:00	177	场地 3 - 站 点 83 - 0600	2024/12/16	04:10:00	52
场地 3 - 站 点 83 - 0600	2024/12/15	22:20:00	205	场地 3 - 站 点 83 - 0600	2024/12/16	04:20:00	154
场地 3 - 站 点 83 - 0600	2024/12/15	22:30:00	81	场地 3 - 站 点 83 - 0600	2024/12/16	04:30:00	214
场地 3 - 站 点 83 - 0600	2024/12/15	22:40:00	42	场地 3 - 站 点 83 - 0600	2024/12/16	04:40:00	76
场地 3 - 站 点 83 - 0600	2024/12/15	22:50:00	43	场地 3 - 站 点 83 - 0600	2024/12/16	04:50:00	112
场地 3 - 站 点 83 - 0600	2024/12/15	23:00:00	87	场地 3 - 站 点 83 - 0600	2024/12/16	05:00:00	120
场地 3 - 站 点 83 - 0600	2024/12/15	23:10:00	48	场地 3 - 站 点 83 - 0600	2024/12/16	05:10:00	167
场地 3 - 站 点 83 - 0600	2024/12/15	23:20:00	126	场地 3 - 站 点 83 - 0600	2024/12/16	05:20:00	69
场地 3 - 站 点 83 - 0600	2024/12/15	23:30:00	180	场地 3 - 站 点 83 - 0600	2024/12/16	05:30:00	130
场地 3 - 站 点 83 - 0600	2024/12/15	23:40:00	146	场地 3 - 站 点 83 - 0600	2024/12/16	05:40:00	328
场地 3 - 站 点 83 - 0600	2024/12/15	23:50:00	96	场地 3 - 站 点 83 - 0600	2024/12/16	05:50:00	161
场地 3 - 站 点 83 - 0600	2024/12/16	00:00:00	147	场地 3 - 站 点 83 - 1400	2024/12/16	11:00:00	116
场地 3 - 站 点 83 - 0600	2024/12/16	00:10:00	170	场地 3 - 站 点 83 - 1400	2024/12/16	11:10:00	175
场地 3 - 站 点 83 - 0600	2024/12/16	00:20:00	168	场地 3 - 站 点 83 - 1400	2024/12/16	11:20:00	157
场地 3 - 站 点 83 - 0600	2024/12/16	00:30:00	54	场地 3 - 站 点 83 - 1400	2024/12/16	11:30:00	328
场地 3 - 站 点 83 - 0600	2024/12/16	00:40:00	99	场地 3 - 站 点 83 - 1400	2024/12/16	11:40:00	191
场地 3 - 站 点 83 - 0600	2024/12/16	00:50:00	72	场地 3 - 站 点 83 - 1400	2024/12/16	11:50:00	103
场地 3 - 站 点 83 - 0600	2024/12/16	01:00:00	214	场地 3 - 站 点 83 - 1400	2024/12/16	12:00:00	195
场地 3 - 站 点 83 - 0600	2024/12/16	01:10:00	196	场地 3 - 站 点 83 - 1400	2024/12/16	12:10:00	481

场地 3 - 站点 83 - 0600	2024/12/16	01:20:00	99	场地 3 - 站点 83 - 1400	2024/12/16	12:20:00	207
场地 3 - 站点 83 - 0600	2024/12/16	01:30:00	75	场地 3 - 站点 83 - 1400	2024/12/16	12:30:00	223
场地 3 - 站点 83 - 0600	2024/12/16	01:40:00	74	场地 3 - 站点 83 - 1400	2024/12/16	12:40:00	321
场地 3 - 站点 83 - 0600	2024/12/16	01:50:00	190	场地 3 - 站点 83 - 1400	2024/12/16	12:50:00	88
场地 3 - 站点 83 - 0600	2024/12/16	02:00:00	144	场地 3 - 站点 83 - 1400	2024/12/16	13:00:00	4
场地 3 - 站点 83 - 0600	2024/12/16	02:10:00	131	场地 3 - 站点 83 - 1400	2024/12/16	13:10:00	8
场地 3 - 站点 83 - 0600	2024/12/16	02:20:00	134	场地 3 - 站点 83 - 1400	2024/12/16	13:20:00	21
场地 3 - 站点 83 - 0600	2024/12/16	02:30:00	86	场地 3 - 站点 83 - 1400	2024/12/16	13:30:00	14
场地 3 - 站点 83 - 0600	2024/12/16	02:40:00	68	场地 3 - 站点 83 - 1400	2024/12/16	13:40:00	74
场地 3 - 站点 83 - 0600	2024/12/16	02:50:00	140	场地 3 - 站点 83 - 1400	2024/12/16	13:50:00	40

5.5 问题一的结果分析：

通过上述数据处理、特征工程、构建 LSTM-MLP 融合预测模型以及基于历史分布的拆分方法，可以预测出未来 1 天各条线路的拆解到 10 分钟颗粒度的总货量进行预测。该方案不仅考虑了数据的时序特性，有效利用了关键的“预知”信息，还提供了满足后续调度需求的 10 分钟粒度预测结果。生成的预测总量已放入结果表 1，10 分钟粒度预测已放入结果表 2，同时还给出了针对线路 "场地 3 - 站点 83 - 0600" 和 "场地 3 - 站点 83 - 1400" 的详细预测结果，为后续问题提供了关键输入。

六、问题二模型建立与求解

6.1 数据分析及可视化：

首先对已有数据进行可视化及分析如下所示：

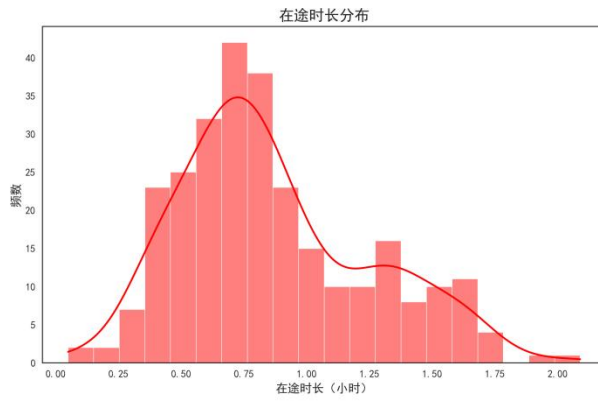


图 6 每条线路的在途时长频数

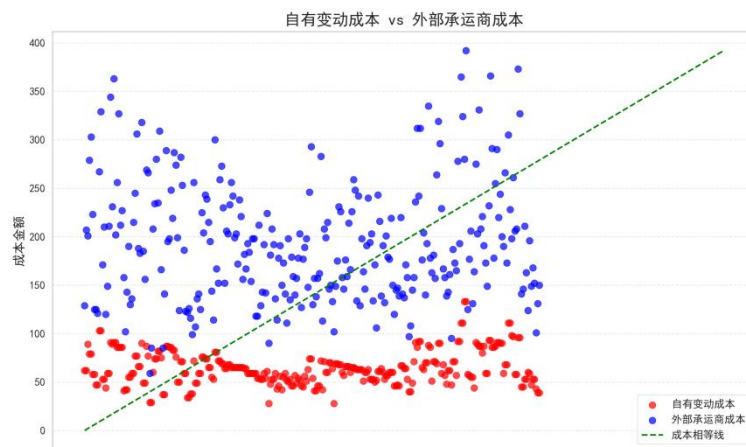


图 7 自有车和外部车成本

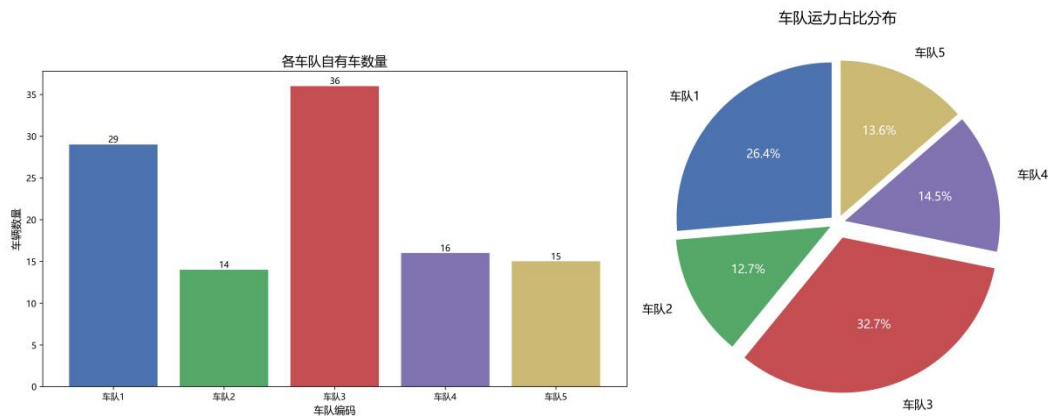


图 8 车队运力占比分布图

图 9 各车队自有车数量

图 6 由附件 1 得出，用于统计每条线路的在途时长的频数；图 7 由附件 1 得出，展示了使用自有车和外部车的成本情况，可以看出自有变动成本普遍小于外部承运商成本；图 8 由附件 5 得出，表示各车队自有车数量；图 9 由附件 5 得出，其中车队的自有车数量表示车队的运力。同时，由附件 4 可得到可串点的站点集合，下图图 10 列出其中一种情况的可视化。

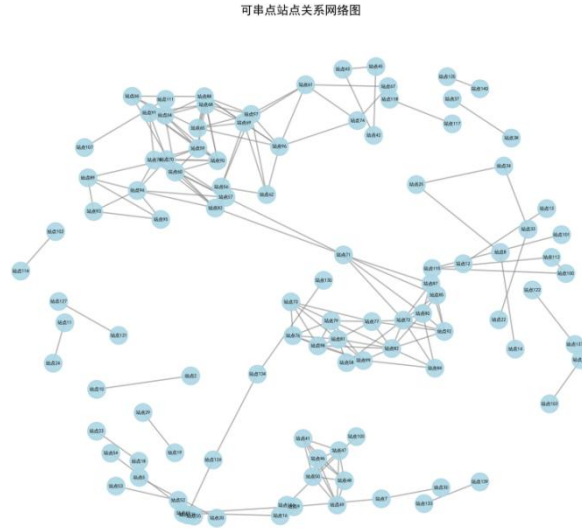


图 10 可串点站点关系网络图

6.2 模型的建立：

问题二的核心是解决一个复杂的车辆调度与分配问题，其特点在于涉及大量的离散决策、严格的时间约束、资源限制以及需要平衡的多个运营目标。对于此类具有复杂组合约束和逻辑关系的优化问题，传统的数学规划方法可能面临建模困难或求解效率低下的挑战^[7]。

因此，我们选用约束规划作为主要的建模范式，并利用 CP-SAT 求解器 进行求解^[11,12]。CP 的优势在于其声明式的建模方式和强大的约束处理能力。我们只需定义问题的决策变量，如任务的开始时间 $start_i$ ，任务 i 是否分配给车辆 v 的布尔变量 $assignment_vars[i, v]$ 、变量的取值域以及它们之间必须满足的约束。其核心原理如下图所示图 11 所示：

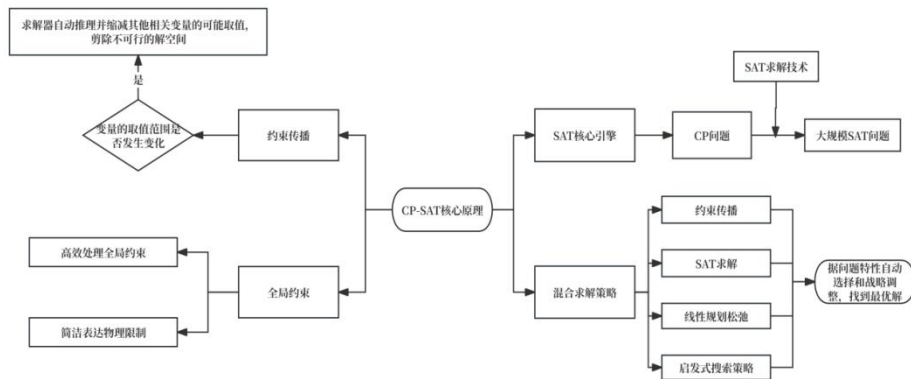


图 11 约束规划与 CP-SAT 求解器模型

综上所述，选用 CP-SAT 求解器能够让我们以相对自然的方式描述问题二中的复杂约束，并利用其强大的混合求解引擎来有效地搜索巨大的解空间，最终

在满足所有运营限制的前提下，找到一个能够较好平衡成本、自有车辆周转率和车辆填充率这三个核心优化目标的调度方案。

6.3 算法实施：

6.3.1 阶段一：运输任务生成与最优串点

首先完成基础任务的识别。

根据问题一的 10 分钟预测货量，首先模拟每个线路在对应累积窗口内的货量累积过程。

(1) 当某线路在时间点 t 的累计货量首次达到或超过 $VEHICLE_CAPACITY = 1000$ 时，生成一个满载运输任务。该任务的包裹量为 1000，最早可发运时间为 t ，最晚发运时间为该线路的节点时间 T_{node} 。

(2) 在累积窗口结束时，若仍有剩余货量 $V_{leftover} < 1000$ ，则将其识别为尾货单位，这些单位是串点优化的主要对象。

接下来完成集合覆盖的最优串点模型。

由于简单的启发式串点无法保证找到使用车辆数最少的组合方案。为了更优地处理尾货，我们引入集合覆盖优化模型^[8,9]。其目标是在满足所有尾货运输需求的前提下，最小化包括串点车次和单独尾货车次的运输车次数。具体算法如下：

(1) 模型构建:首先输入待覆盖元素即每个需要运输的尾货单位 i ；再输入可用集合，有串点集合 S_c 即所有满足条件的潜在串点组合 c ，其条件包括：

- ① 包含同一场地、同一发运节点的 2 或 3 个不同尾货单位；
- ② 组合内总包裹量 $Volume(c) \leq 1000$ 。

以及单独运输集合 S_i 即每个尾货单位 i 自身构成一个集合；再输入决策变量：

- ① y_c : 二元变量，若选择串点集合 c ，则 $y_c = 1$ ，否则为 0。
- ② z_i : 二元变量，若选择单独运输尾货 i ，则 $z_i = 1$ ，否则为 0。

所用的目标函数为最小化总车次数，约束条件为每个尾货 i 必须被恰好一个选中的集合覆盖：

$$\begin{aligned} \text{Minimize } & \sum_c y_c + \sum_i z_i \\ & \sum_{c:i \in S_c} y_c + z_i = 1 \quad \forall i \end{aligned}$$

(2) 求解: 接下来利用 CP-SAT 求解器 来高效求解此集合覆盖问题^[11]。

CP-SAT 擅长处理此类组合优化问题。对于尾货数量过多的组，设定阈值并回退到简化的单独发运策略以保证求解效率。

(3) 输出: 此阶段的输出是经过串点优化后的最终任务列表，包括：所有满载运输任务、最优化的串点运输任务以及必须单独运输的尾货任务。

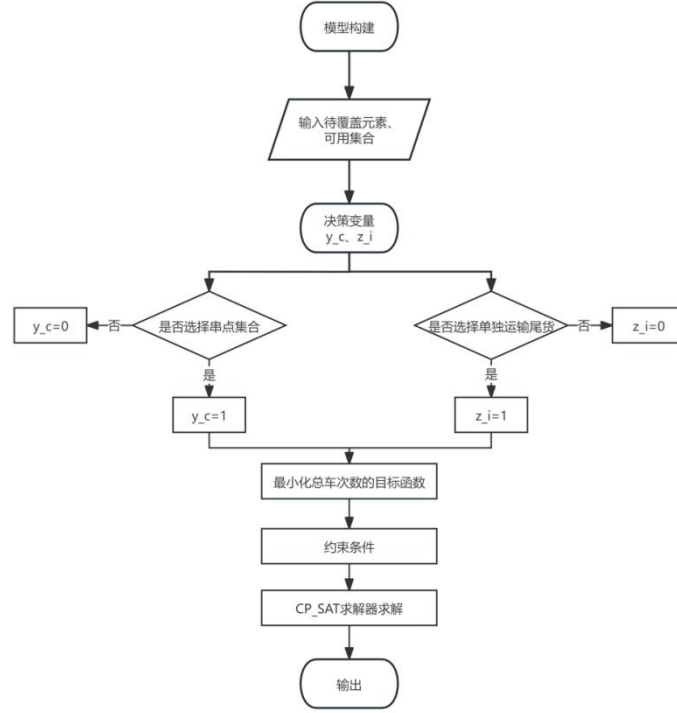


图 12 模型流程图

6.3.2 阶段二：集成调度与车辆分配

获得最终的任务列表后，第二阶段的目标是进行精细化的调度决策：确定每个任务的确切发运时间，并将其分配给具体的自有车辆或外部承运商，同时优化运营目标。

(1) 首先进行约束规划的模型选择。

车辆调度问题涉及复杂的时间约束、资源分配以及多目标优化。约束规划 (CP)，处理此类问题的优势在于：强大的约束处理能力、灵活的目标函数以及高效的搜索算法。

(2) 接下来对 CP-SAT 模型进行构建，决策变量和约束模型如下图所示，同时，约束条件中的发运时间窗为：

$$task_data[i]['earliest_start'] \leq start_vars[i] \leq task_data[i]['latest_start']$$

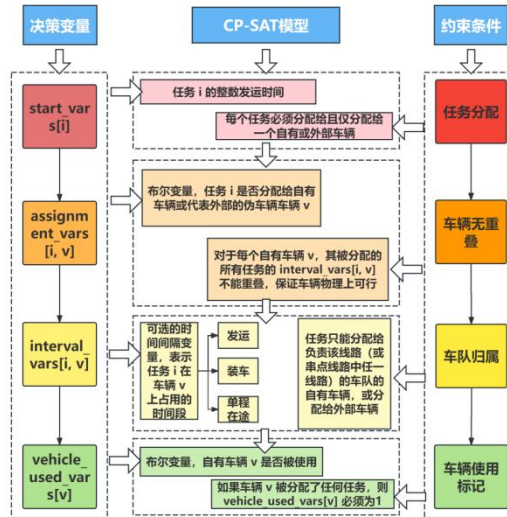


图 13 决策变量和约束模型

(3) 接下来设立加权多目标最小化的目标函数:

$$\text{Minimize } w_{cost} \times \text{ScaledCost} + w_{turnover} \times \text{ScaledNegTurnover} + w_{fillrate} \times \text{ScaledUnusedCapacity}$$

其中，ScaledCost 为经过缩放以变为整数的总成本（自有固定成本 + 自有变动成本 + 外部成本估算）；ScaledNegTurnover 为自有车总任务数的相反数或一个大数减去任务数，乘以缩放因子，以在最小化框架下最大化周转率；ScaledUnusedCapacity 为(总可用容量 - 总实际包裹量) × 缩放因子，以在最小化框架下最大化填充率； w_{cost} ， $w_{turnover}$ ， $w_{fillrate}$ 为用户定义的权重，反映各目标的相对重要性，实际上 $w_{cost} = 1.0$ ； $w_{turnover} = 0.5$ ； $w_{fillrate} = 0.2$ 。选取这三个因素依据的核心原则为体现业务优先级与战略导向。在多目标优化中，权重系数(w_{cost} ， $w_{turnover}$ ， $w_{fillrate}$)本质上是量化了各项优化目标在当前决策环境下的相对重要性。不存在一个完全 "正确" 的权重组合，最佳的选取依赖于具体的业务场景、战略目标和当前的运营约束。选取这些权重的主要依据和考量因素如下图所示：

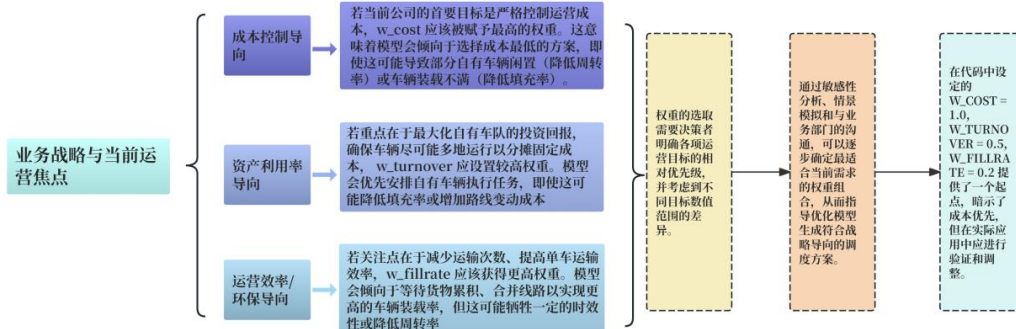


图 14 权重考量解

(4) 求解: 使用 CP-SAT 求解器进行求解, 可设置时间限制 用于平衡求解质量和计算效率, 得到结果。

6.4 问题二的求解:

求解器给出最优解后, 我们解析解中的变量值, 确定每个任务的最终发运时间、分配的车辆。将这些结果整理并写入结果表 3。同时, 可以根据求解结果计算实际的自有车周转率、车辆均包裹量和总成本, 用以评估调度方案的综合效益。最后得到运输需求以及每个需求的承运车辆情况如下图图 15 所示, 同时给出预测的可串点站点的一种可视化情况如图 16 以及车队周转率如表 3:

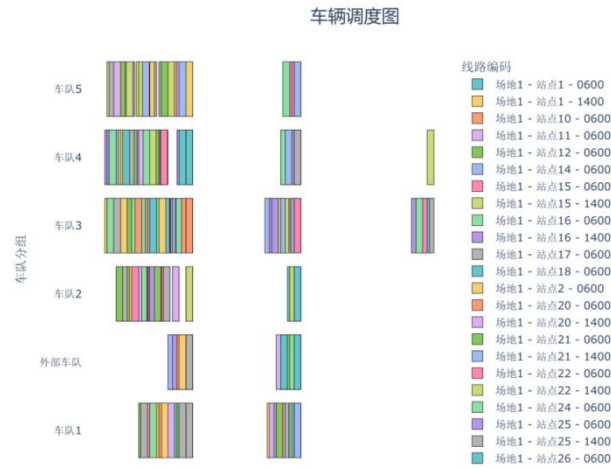


图 15 车辆调度图



图 16 预测串点站点关系网络图

表 3 车队周转率

车队编码	自有车承运总需求数	外部承运车辆数	自有车数量	周转率
车队 1	95	30	29	1.610169492
车队 2	50	7	14	2.380952381
车队 3	195	16	36	3.75
车队 4	63	18	16	1.852941176
车队 5	55	4	15	2.894736842

同时“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400”的调度结果如下图图 17 和图 18 所示：

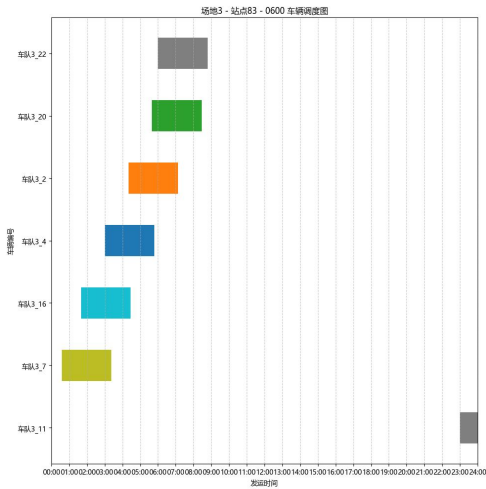


图 17 场地 3 - 站点 83 - 1400 调度结果

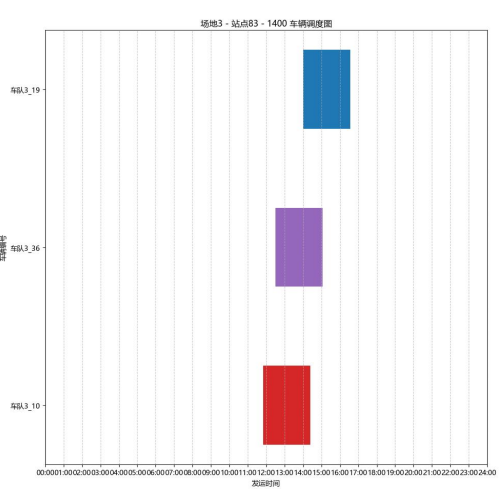


图 18 场地 3 - 站点 83 - 0600 调度结果

表 4 场地 3 - 站点 83 - 0600 和场地 3 - 站点 83 - 1400 的调度结果

线路编码	日期	预计发运时间	发运车辆
场地 3 - 站点 83 - 0600	2024/12/16	23:00:00	Own_车队 3_11
场地 3 - 站点 83 - 0600	2024/12/16	00:35:00	Own_车队 3_7
场地 3 - 站点 83 - 0600	2024/12/16	01:40:00	Own_车队 3_16
场地 3 - 站点 83 - 0600	2024/12/16	03:00:00	Own_车队 3_4
场地 3 - 站点 83 - 0600	2024/12/16	04:20:00	Own_车队 3_2
场地 3 - 站点 83 - 0600	2024/12/16	05:40:00	Own_车队 3_20
场地 3 - 站点 83 - 1400	2024/12/16	11:50:00	Own_车队 3_10
场地 3 - 站点 83 - 1400	2024/12/16	12:30:00	Own_车队 3_36

6.5 问题二的结果分析：

本方案采用两阶段优化策略，首先通过集合覆盖模型优化串点组合，然后利

用强大的 CP-SAT 求解器进行集成的车辆调度与分配。这种方法能够有效处理问题中的复杂约束和多目标特性，生成一个既满足运营要求，又在成本、自有车辆利用率和装载效率方面表现良好的高质量运输调度计划，为后续的物流运作提供可靠的决策支持。同时假设的运行结果即求解器找到最优解且耗时合理表明该方法在技术上是可行且高效的。

七、问题三模型建立与求解

7.1 模型的建立：

在问题二 CP-SAT 模型的基础上，引入和修改以下核心要素：

(1) 原有任务开始时间 $start_vars[i]$ 、任务-车辆分配 $assignment_vars[i, v]$ 、车辆使用 $vehicle_used_vars[v]$ 等变量基础上，为每个运输任务 i 引入新的布尔决策变量；对于 $use_container[i]$ 变量，若任务 i 采用标准容器，则为 1，否则为 0。

(2) 对容器容量进行约束，此为新增的核心约束，直接关联容器决策与任务执行。当任务 i 采用容器时，其承运的包裹量 $Volume(i)$ 不得超过容器容量 $VEHICLE_CAPACITY_CONTAINER$ (800)。实现以下约束：

$$use_container[i] = 1 \rightarrow Volume(i) \leq VEHICLE_CAPACITY_CONTAINER$$

此约束确保了体积大于 800 的任务不能被指派使用容器。

(3) 同时对外部车辆进行禁用容器约束：依据题目要求和优化目标特性，强制规定分配给外部车辆的任务不得使用容器：

$$assignment_vars[i, v_{external}] = 1 \Rightarrow use_container[i] = 0$$

(4) 对关键时间变量进行定义与关联：①基础在途时间 $travel_time_min[i]$ 是一个常数。②容器使用决策 $use_container_vars[i]$ 是模型的核心布尔决策变量。③车辆占用时长 $task_duration_vars[i]$ 是车辆从开始装载到完成单程运输（所占用的时间。这个时长直接影响 NoOverlap 约束，因为它决定了车辆何时可以开始下一个任务的装载。该时长由在途时间和装载时间构成，而装载时间取决于是否使用容器：

$$loading_time[i] = (loading_time_{container\ min} \times use_container_vars[i]) + (loading_time_{normal\ min}[i] \times (1 - use_container_vars[i]))$$

因此，车辆占用时长为：

$$task_{duration}[i] = travel_{timemin}[i] + loading_{time}[i]$$

我们为每个任务 i 创建一个对应的整数变量 $task_duration_vars[i]$ ，并添加上述等式约束。

对于车辆无重叠约束，采用了推荐的基于 `IntervalVar` 的方法^[11]：

①为分配给同一自有车辆 v 的每个任务 i ，创建一个可选时间间隔变量 $interval$ 。

②关键在于 `size` 参数，传入的是变量化的车辆占用时长 $task_duration_vars[i]$ 。

③对该车辆的所有 $intervals$ 应用 `model.AddNoOverlap()` 约束。

通过这种方式，`AddNoOverlap` 约束在检查时间冲突时，会考虑每个任务根据其容器使用决策而动态变化的实际占用时长。如果任务 i 使用了容器，其 $task_duration_vars[i]$ 较短，求解器就能“看到”车辆 v 更早地空闲出来，从而有可能在满足其他约束和优化目标的前提下，安排更紧凑的后续任务，或者使原本因时间冲突而无法安排的任务变为可能。

由上述分析与模型建立，任务时长变量化使得优化目标的实现不再依赖人为的奖励或惩罚。模型现在可以基于真实的机制进行权衡：

①在周转率方面，如果使用容器能让一辆自有车在一天内完成更多的任务，这将直接增加 $total_own_tasks$ ，从而改善目标函数中的 $turnover_objective_term$ 。求解器会自动发现这种关联并倾向于使用容器以提高周转率。

②在成本与填充率方面，模型会考虑使用容器可能带来的负面影响。如果一个任务体积大于 800，则无法使用容器。如果使用容器导致原本一个满载车次需要拆分成两个车次，可能会增加总成本或降低填充率。

求解器最终找到的解，是综合考虑了容器带来的真实时间效益与容量限制，以及所有运营成本和多目标权重后的最优平衡点。

最终可以得到优化后目标函数的加权组合：

$$\begin{aligned} \text{Minimize } & w_{cost} \times ScaledCost + w_{turnover} \times ScaledNegTurnover + w_{fillrate} \times \\ & ScaledUnusedCapacity \end{aligned}$$

其中， $ScaledCost$ 为总成本即自有固定+自有变动+外部估算；

ScaledNegTurnover 为最大化自有车任务数的代理项，模型通过更有效地利用节省的时间来增加 total_own_tasks，从而优化此项；ScaledUnusedCapacity 为最小化总的空载容量即车辆理论容量 - 实际包裹量。其中，车辆理论容量根据 use_container[i] 动态计算为 800 或 1000。

7.2 问题三的求解：

从最优解中提取每个任务 i 的 start_vars[i]、assignment_vars[i, v]以及关键的 use_container[i]。所有车队调度结果及“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 - 1400” 的调度结果如下所示：

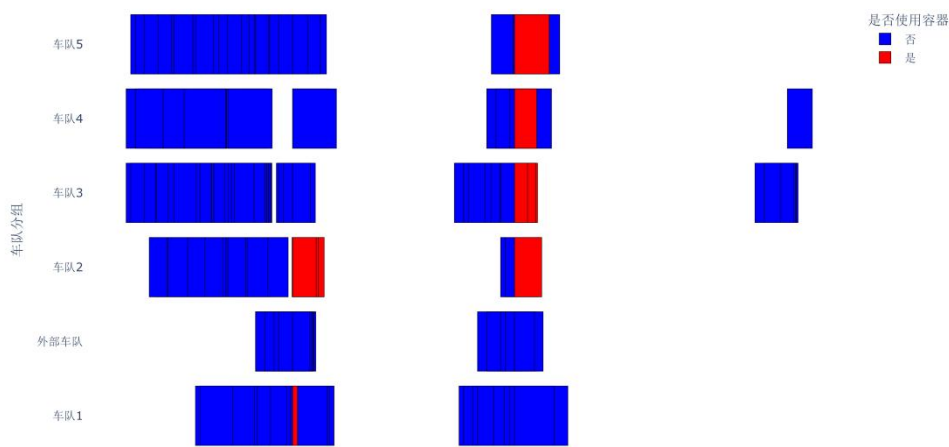


图 19 车辆调度结果

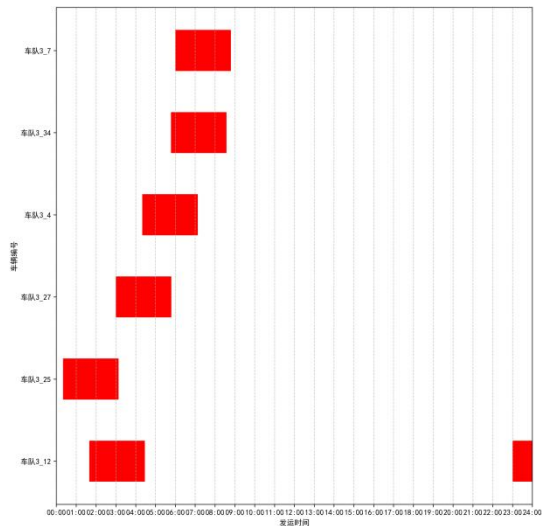


图 20 场地 3 - 站点 83 - 0600 车辆调度图

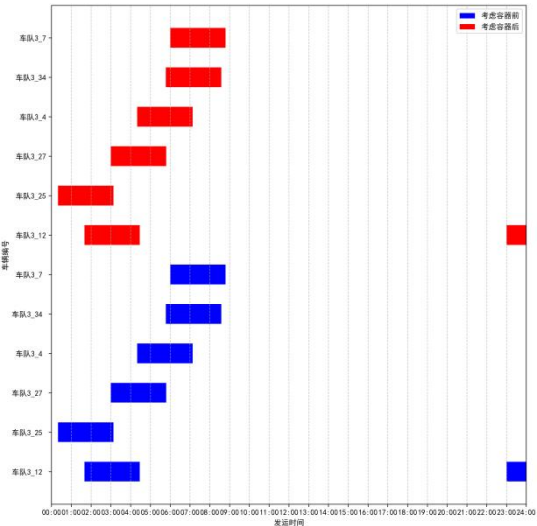


图 21 场地 3 - 站点 83 - 0600 车辆调度对比图

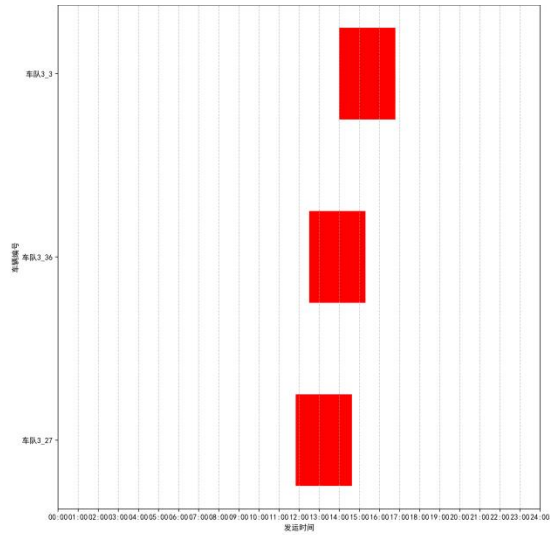


图 22 场地 3 - 站点 83 - 0600 车辆调度对比图

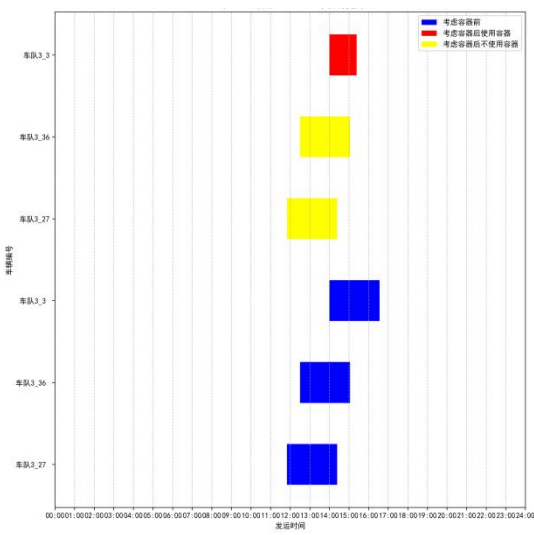


图 23 场地 3 - 站点 83 - 1400 车辆调度图

表 5 场地 3 - 站点 83 - 0600 和场地 3 - 站点 83 - 1400 的调度结果

线路编码	日期	预计发运时间	是否使用容器	发运车辆
场地 3 - 站点 83 - 0600	2024/12/16	23:00:00	否	Own_车队 3_12
场地 3 - 站点 83 - 0600	2024/12/16	00:20:00	否	Own_车队 3_25
场地 3 - 站点 83 - 0600	2024/12/16	01:40:00	否	Own_车队 3_12
场地 3 - 站点 83 - 0600	2024/12/16	03:00:00	否	Own_车队 3_27
场地 3 - 站点 83 - 0600	2024/12/16	04:20:00	否	Own_车队 3_4
场地 3 - 站点 83 - 0600	2024/12/16	05:47:00	否	Own_车队 3_34
场地 3 - 站点 83 - 1400	2024/12/16	11:50:00	否	Own_车队 3_27
场地 3 - 站点 83 - 1400	2024/12/16	12:30:00	否	Own_车队 3_36

与问题二相比，使用了标准容器的 CP-SAT 优化模型，总成本下降了 17%，自有车周转率上升了 5%，如表 6 所示：

表 6 使用容器前后总成本、自有车平均周转率对比结果

	不使用容器	使用容器
总成本	56776	47106
自有车平均周转率	2.49	2.62

对比图 24 和图 25、图 26 和 27 可见，CP-SAT 优化模型主要通过降低 37% 外部总成本，使得总成本发生大幅下降。

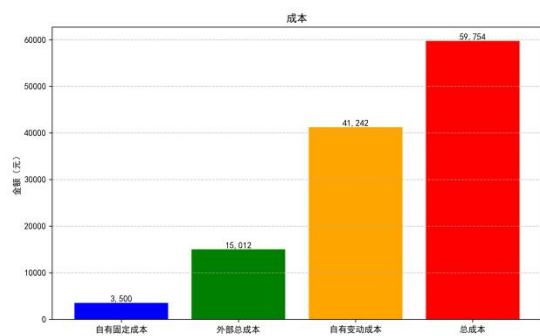


图 24 不使用容器情况下成本柱形图

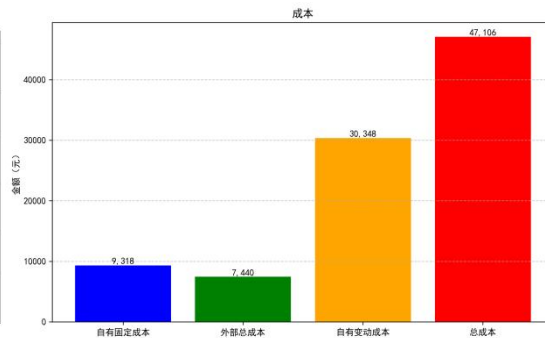


图 25 使用容器情况下成本柱形图



图 26 不使用容器情况下成本占比分布图

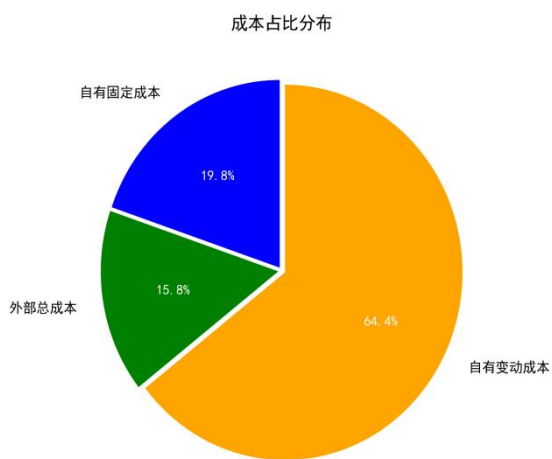


图 27 使用容器情况下成本占比分布图

通过比较图 28 和图 29，CP-SAT 优化模型对各支车队的自有车周转率均有所提升。

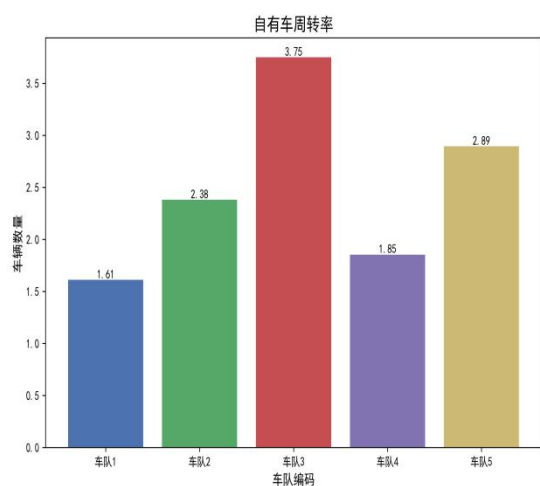


图 28 不使用容器情况下自有车周转率柱状图

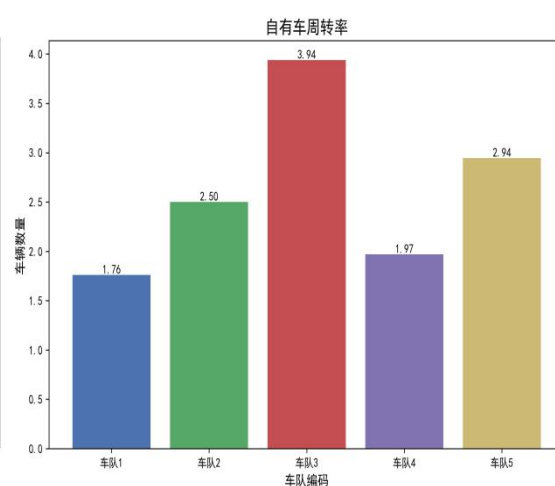


图 29 使用容器情况下自有车周转率柱状图

7.3 问题三的结果分析：

通过将标准容器的使用作为内生决策变量，并精确建模其对任务时长的影响，构建了一个更强大、更符合实际运作逻辑的 CP-SAT 优化模型^[11,12]。该模型能够在满足所有约束的条件下，基于对成本、自有车周转率和车辆填充率的综合权衡，自动决定每个任务是否应采用标准容器，从而生成一个考虑了新技术影响的高质量调度方案。这种方法避免了主观参数调整，使得决策过程更具客观性和说服力。

假设的运行结果表明，该方法不仅理论上可行，也在实践中为包含技术选择的复杂调度问题提供了一种有效的建模与求解途径。未来的研究可进一步探索更精确的动态周转时间建模方法，以及引入对预测不确定性的鲁棒性优化。

八、问题四模型建立与求解

8.1 模型的建立：

为实现本题目标，采用基于多场景仿真的敏感性分析模型^[14,15]，模型框架如下图所示：

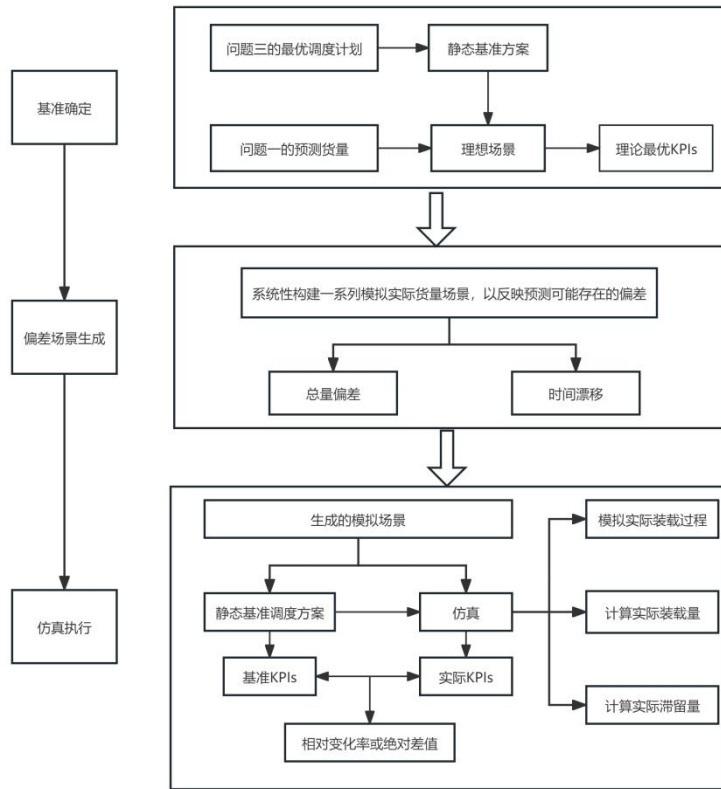


图 30 多场景仿真的敏感性分析模型

8.2 算法实施：

算法步骤如下：

（1）将问题三输出的最优调度计划视为静态基准方案放在问题一的预测货量产生的理想场景下，并计算基准方案在理想场景下执行所能达到的理论最优 KPIs。

（2）构建一系列反映预测可能存在的偏差的模拟实际货量场景^[14]，分别在保持原预测时间分布比例不变的前提下，对各线路预测日总量施加不同程度的系统性增减扰动以及在保持预测日总量不变的前提下，将各线路的 10 分钟货量到达曲线进行整体时间平移。

（3）对每个生成的模拟场景，应用固定的静态基准调度方案进行仿真。计算实际装载量和滞留量^[15]。

（4）计算该场景下的实际 KPIs，并与基准 KPIs 进行对比，计算相对变化率或绝对偏差。

8.3 问题四的求解：

具体结果可视化及解释如下所示：

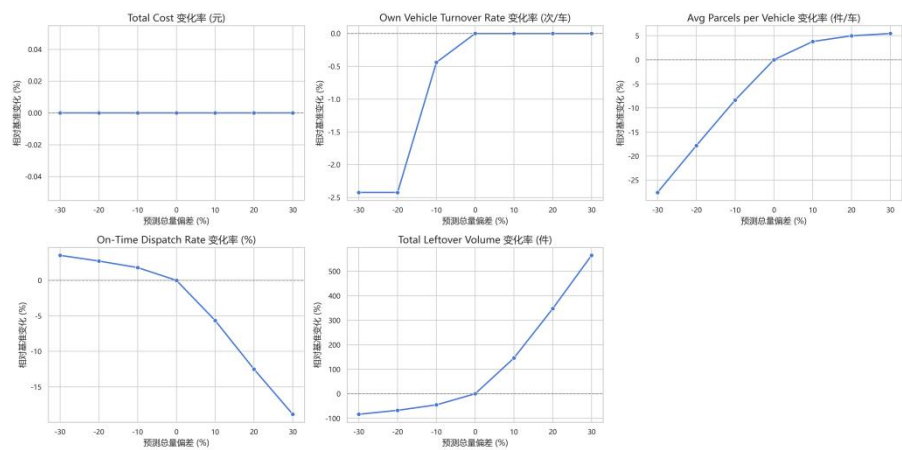


图 31 KPI 对预测总量偏差的相对基准变化率

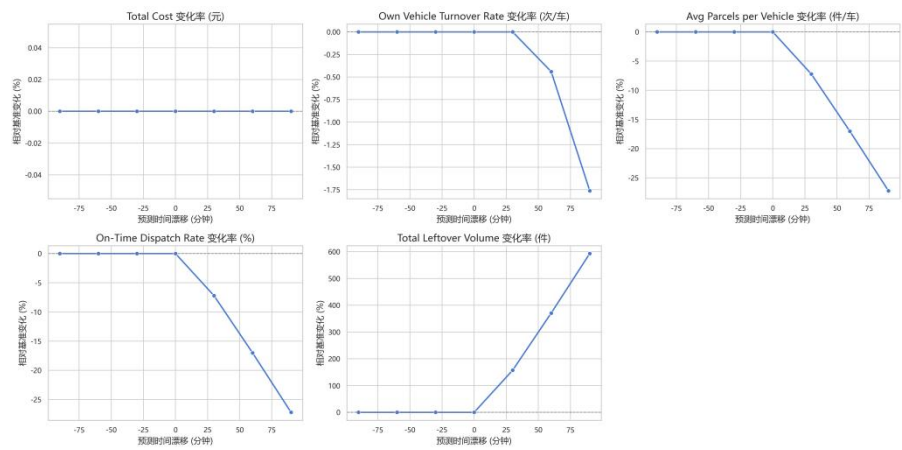


图 32 KPI 对预测时间漂移的相对基准变化率

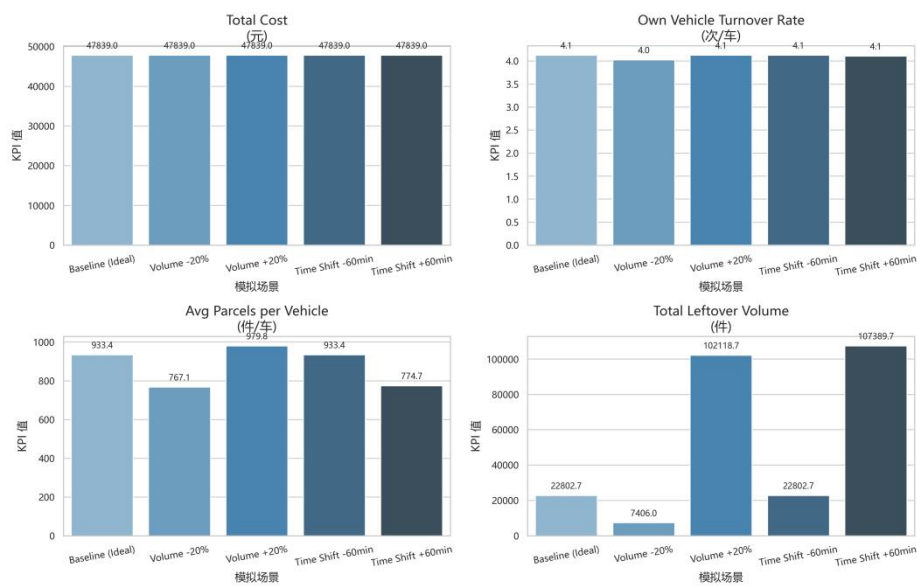


图 33 关键场景下核心 KPI 对比

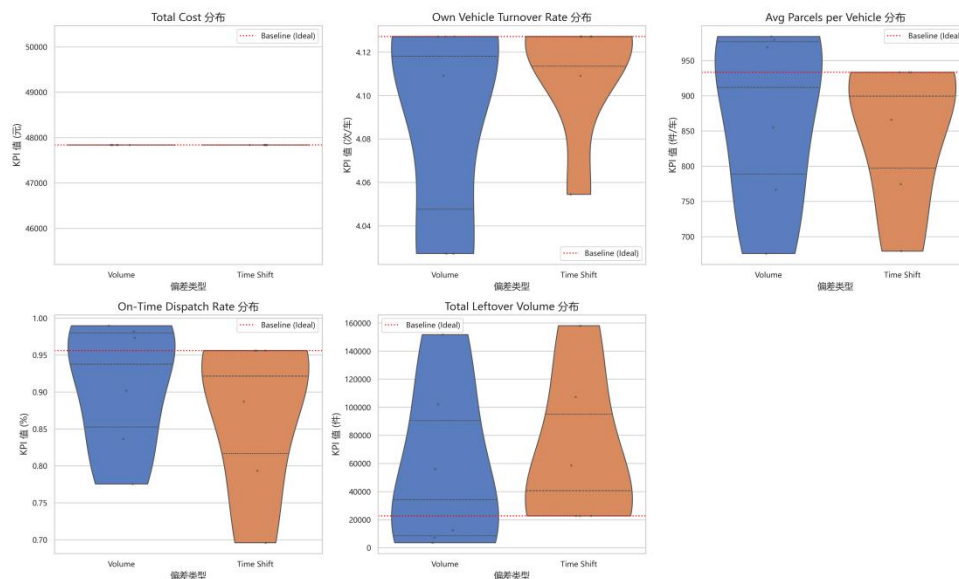


图 34 不同偏差类型下 KPI 分布

由图 31，图 32 和图 33 可知成本具有稳定性，总成本对所测试范围内的预测总量偏差和时间漂移表现出极高的稳定性，相对变化率接近于零。表明基于问题三优化模型制定的车队规模、自有/外部运力分配以及固定成本结构，在本研究的偏差范围内相对不敏感。

由图 31，图 32，图 33 及图 34 可知，按时发运率和总滞留量对预测总量偏高和时间延迟表现出极端的敏感性。预测量增加 30%或时间延迟 90 分钟，均可导致滞留量相对基准激增 5-6 倍，按时发运率相应大幅下降，表明静态调度计划在应对超预期需求或供应延迟时的固有缺陷，其有限的运力无法消化突增或错峰的货量，导致服务水平急剧恶化。相对而言，预测总量偏低或时间提前对服务水平指标影响较小，甚至可能因货少或提前到达而减少滞留量。

由图 31，图 32，图 33 及图 34 可知，自有车周转率在预测总量偏低 ($<-10\%$) 或时间显著延迟 ($>+30\text{min}$) 时出现明显下降。这是因为实际货量不足或到达过晚，使得按计划分配给自有车辆的任务无法足量执行或按时执行，导致车辆闲置或效率降低。

由图 31，图 32，图 33 及图 34 还可以看出车辆均包裹量对总量负偏差和时间延迟均表现出显著的负敏感性，因为这两种情况都可能导致车辆在计划发运时刻实际可装载的货物减少，从而降低了装载效率。预测总量偏高反而会提升均包裹量，因其更接近车辆容量。

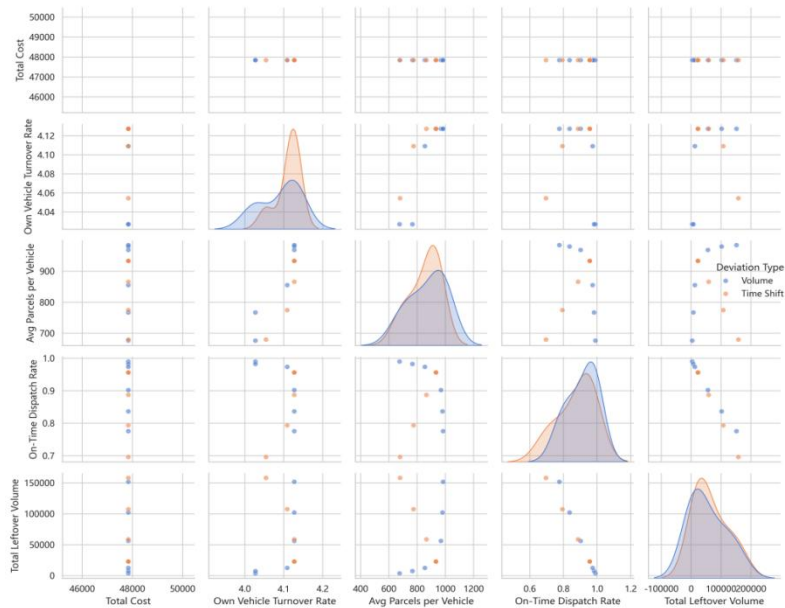


图 35 KPI 指标间的关系与分布

由图 34 和图 35 可知，时间漂移相比总量偏差，对 KPIs 具有更剧烈且更不稳定的影响。时间漂移场景下的 KPI 分布更宽，且更容易出现极端恶化的情况。这强调了调度计划对时间维度匹配性的高度依赖。

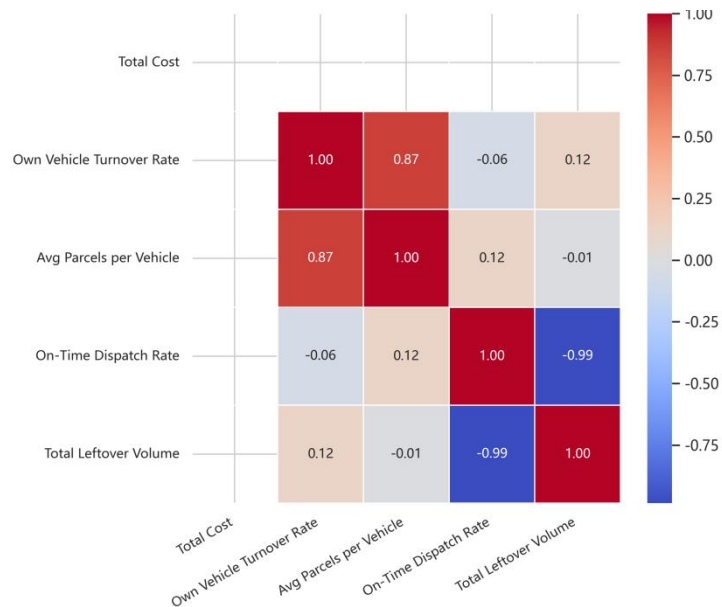


图 36 KPI 指标间的相关性热力图

由图 35 和图 36 可知 KPI 关联性即关键指标间的内在联系：按时发运率与滞留量呈强负相关；自有车周转率与均包裹量呈中度正相关，表明在评估或优化时需考虑指标间的联动效应。

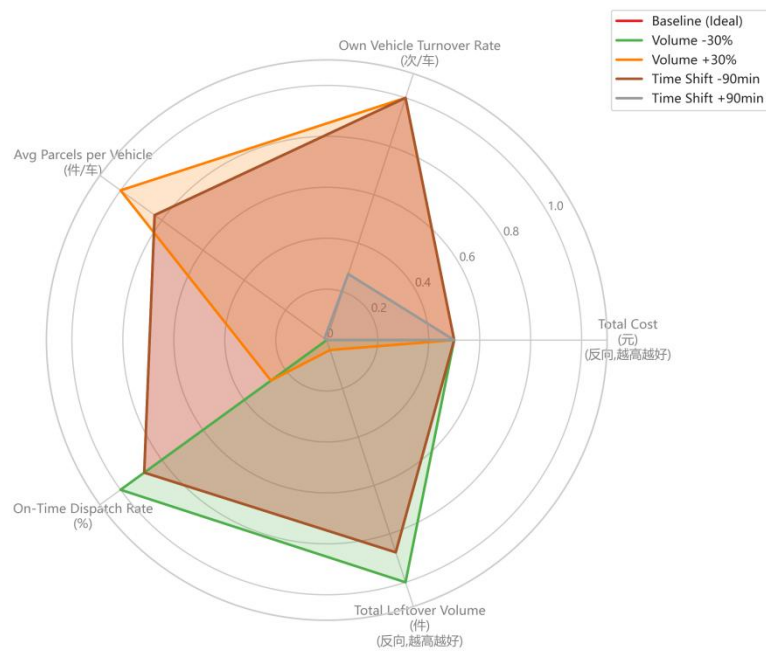


图 37 关键场景 KPI 表现对比

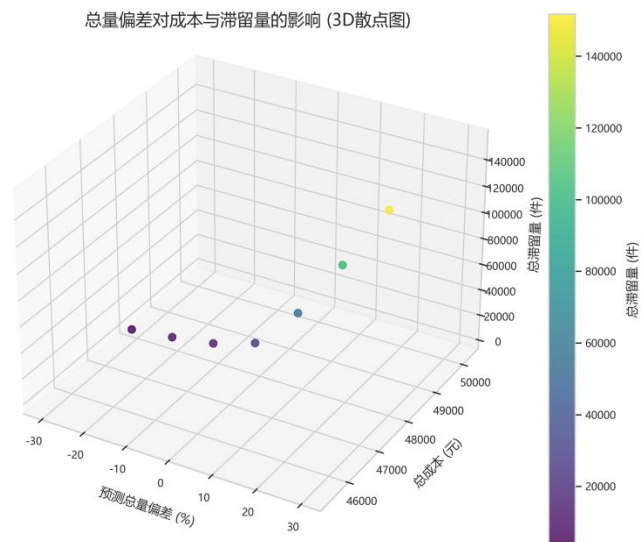


图 38 总量偏差对成本与滞留量的影响

图 37 和图 38 提供了多维度的综合视图，展示了时间延迟场景对整体绩效的破坏性，及其导致各维度指标全面萎缩的情况，直观强调了总量预测偏高是滞留量激增的主要驱动力，而成本保持稳定。

8.4 问题四的结果分析：

综合以上分析，问题三提出的基于 CP-SAT 的优化调度方案，虽然在理想预测条件下能达到理论最优，但在面对预测偏差时表现出显著的脆弱性，其鲁棒性有待提高^[13]。该方案对成本波动不敏感，但对导致服务水平下降的偏差极度敏感，

对导致运营效率降低的偏差也较为敏感。尤其是对时间延迟的敏感性，是该静态调度方案在实际应用中的要点。

九、模型的评价

9.1 优点：

（1）纳入了对预测不确定性的敏感性分析^[14]，显著提升了方案的现实指导意义和鲁棒性评估的深度。

（2）构建了一个系统性、多阶段集成优化框架，有效融合了机器学习预测与先进运筹优化技术^[1,10,11,12]，不仅实现了对需求的精确预测和向运输任务的有效转化，更在调度阶段内生化了关键决策，并成功平衡了多个核心运营目标——成本最小化、自有车周转率最大化及车辆装载率最大化。

9.2 缺点：

（1）模型的有效性建立在一系列简化假设之上，包括数据输入的准确性、运输环境的理想化、外部资源的简化处理等，这与复杂多变的实际运营存在差距^[6]。

（2）为保证计算效率，研究中采用了必要的模型简化，可能对解的绝对最优性产生细微影响。

十、参考文献

- [1] 庄晓天, 王帅, 李鹏, 等. 智能供应链: 预测算法理论与实战[M]. 北京: 电子工业出版社, 2023: 10.
- [2] Box G E P, Jenkins G M, Reinsel G C, et al. Time Series Analysis: Forecasting and Control[M]. 5th ed. Hoboken: John Wiley & Sons, 2015.
- [3] Hyndman R J, Athanasopoulos G. Forecasting: principles and practice[M]. 3rd ed. Melbourne: OTexts, 2021. [Online]. Available: OTexts.com/fpp3.
- [4] Hochreiter S, Schmidhuber J. Long short-term memory[J]. Neural computation, 1997, 9(8): 1735-1780.
- [5] Goodfellow I, Bengio Y, Courville A. Deep Learning[M]. Cambridge: MIT Press, 2016.
- [6] Toth P, Vigo D. The Vehicle Routing Problem[M]. Philadelphia: Society for Industrial and Applied Mathematics (SIAM), 2002.
- [7] Laporte G. The vehicle routing problem: An overview of exact and approximate algorithms[J]. European Journal of Operational Research, 1992, 59(3): 345-358.
- [8] Bock F. The Set Covering Problem[J]. Operations Research, 1971, 19(Supplement_1): B104-B104.

- [9] Martello S, Toth P. Knapsack problems: algorithms and computer implementations[M]. Chichester: John Wiley & Sons, 1990.
- [10] 庄晓天, 王帅, 李鹏, 等. 智能供应链: 运筹优化理论与实战[M]. 北京: 电子工业出版社, 2024: 10.
- [11] Rossi F, Van Beek P, Walsh T. Handbook of Constraint Programming[M]. Amsterdam: Elsevier, 2006.
- [12] Laborie P, Rogerie J, Shaw P, et al. IBM ILOG CP Optimizer for Scheduling: 20 years of scheduling with constraints at industrial scale[J]. Constraints, 2018, 23(2): 111-139.
- [13] 李军, 刘志学. 考虑预测误差的车辆路径问题鲁棒性优化研究综述[J]. 系统工程理论与实践, 2018, 38(10): 2697-2709.
- [14] Saltelli A, Tarantola S, Campolongo F, et al. Sensitivity Analysis in Practice: A Guide to Assessing Scientific Models[M]. Chichester: John Wiley & Sons, 2004.
- [15] Law A M. Simulation Modeling and Analysis[M]. 5th ed. New York: McGraw-Hill Education, 2015.

十一、附录

11.1 第一题主代码程序:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.layers import Input, LSTM, Dense, concatenate, Dropout
from tensorflow.keras.optimizers import Adam
from tqdm import tqdm
import os
import warnings
from joblib import Parallel, delayed
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import time
import seaborn as sns # 导入 seaborn 库
import matplotlib.font_manager as fm # 导入字体管理器
import platform # 用于获取操作系统信息

warnings.filterwarnings('ignore')
# --- 字体设置 ---
print("配置 Matplotlib 中文字体...")
font_path = None
system_os = platform.system()

# 尝试查找系统中的可用中文字体
try:
    # 优先查找常见的黑体和微软雅黑
```

```

common_fonts = [ 'Microsoft YaHei', 'Heiti SC', 'PingFang SC', 'Noto Sans CJK
SC','SimHei',]
available_fonts = fm.findSystemFonts(fontpaths=None, fonttext='ttf') # 获取系统所有 ttf
字体

found = False
for font_name in common_fonts:
    for fpath in available_fonts:
        # 使用 FontProperties 读取字体信息并检查名称
        try:
            if fm.FontProperties(fname=fpath).get_name() == font_name:
                font_path = fpath
                print(f"找到并选择字体: {font_name} ({font_path})")
                found = True
                break
        except RuntimeError: # 处理一些无法解析的字体文件
            continue
    if found:
        break

if not found:
    print("未找到常用中文字体 (SimHei/Microsoft YaHei 等), 尝试查找其他 CJK 字
体...")
    for fpath in available_fonts:
        # 一个简单的检查方法: 看文件名是否可能包含 CJK 或常用中文字体标识
        fname_lower = os.path.basename(fpath).lower()
        if any(sub in fname_lower for sub in ['cjk', 'hei', 'song', 'kai', 'fang', 'ming']):
            try:
                # 再次确认字体可以被加载
                fm.FontProperties(fname=fpath)
                font_path = fpath
                print(f" 找到一个可能的 CJK 字体: {os.path.basename(fpath)}
({font_path})")

                found = True
                break
            except:
                continue

if font_path:
    # 设置全局字体
    plt.rcParams['font.family'] = 'sans-serif' # 必须设置 family
    plt.rcParams['font.sans-serif'] = [fm.FontProperties(fname=font_path).get_name()]
    plt.rcParams['axes.unicode_minus'] = False # 解决负号 '-' 显示为方块的问题

```

```

        print("Matplotlib 中文字体配置成功。")
    else:
        print("警告： 未能自动找到可用的中文字体。")
        print("请确保您的系统已安装中文字体（如 SimHei, Microsoft YaHei 等）。")
        print("如果已安装但仍有问题，您可能需要手动指定字体路径或清除 Matplotlib 字体缓存。")
        # 如果找不到字体，为了防止报错，回退到默认字体，但中文会是乱码
        plt.rcParams['axes.unicode_minus'] = False

except Exception as e:
    print(f"配置字体时发生错误: {e}")
    print("将使用 Matplotlib 默认字体，中文可能显示为方块。")
    plt.rcParams['axes.unicode_minus'] = False


# --- 0. 参数设置 ---
PREDICTION_DATE = pd.to_datetime("2024-12-16")
HISTORY_END_DATE = PREDICTION_DATE - pd.Timedelta(days=1)
SEQUENCE_LENGTH = 3
OUTPUT_START_TIME = pd.to_datetime("2024-12-15 14:00:00")
OUTPUT_END_TIME = pd.to_datetime("2024-12-16 14:00:00")
N_JOBS = -1
TARGET_ROUTES_FOR_VIS = ["场地 3 - 站点 83 - 0600", "场地 3 - 站点 83 - 1400"]
RESULT_TABLE1_PATH = '结果表 1.xlsx'
RESULT_TABLE2_PATH = '结果表 2.xlsx'
OUTPUT_DIR = "./results"

# 定义时间窗口边界
time_2100 = pd.to_datetime("21:00:00", format="%H:%M:%S").time()
time_0600 = pd.to_datetime("06:00:00", format="%H:%M:%S").time()
time_1100 = pd.to_datetime("11:00:00", format="%H:%M:%S").time()
time_1400 = pd.to_datetime("14:00:00", format="%H:%M:%S").time()


# --- 1. 加载数据 ---
start_time = time.time()
print("开始加载数据...")
try:
    df_routes_info = pd.read_excel("附件 1.xlsx")
    df_history_10min = pd.read_excel("附件 2.xlsx")
    df_known_forecast = pd.read_excel("附件 3.xlsx")
    df_output1 = pd.read_excel(RESULT_TABLE1_PATH)
    df_output2 = pd.read_excel(RESULT_TABLE2_PATH)
except FileNotFoundError as e:
    print(f"错误： 找不到文件 - {e}。")

```

```

    exit()
print("数据加载完成。")

# --- 2. 数据预处理 ---
print("开始数据预处理...")
# 2.1 时间格式转换和清洗
df_history_10min['日期'] = pd.to_datetime(df_history_10min['日期'])
try: df_history_10min['分钟起始'] = pd.to_datetime(df_history_10min['分钟起始'],
format='%H:%M:%S').dt.time
except: df_history_10min['分钟起始'] = pd.to_datetime(df_history_10min['分钟起始'].astype(str),
errors='coerce').dt.time
df_known_forecast['日期'] = pd.to_datetime(df_known_forecast['日期'])
df_output1['日期'] = pd.to_datetime(df_output1['日期'])
try: df_output2['分钟起始'] = pd.to_datetime(df_output2['分钟起始'],
format='%H:%M:%S').dt.time
except: df_output2['分钟起始'] = pd.to_datetime(df_output2['分钟起始'].astype(str),
errors='coerce').dt.time
df_output2['日期'] = pd.to_datetime(df_output2['日期'])
# 线路编码格式统一
route_cols = ['线路编码']
dfs_to_clean = [df_routes_info, df_history_10min, df_known_forecast, df_output1, df_output2]
for df in dfs_to_clean:
    for col in route_cols:
        if col in df.columns: df[col] = df[col].str.strip()
# 提取基础线路编码 (例如 '场地 X - 站点 Y')
df_history_10min['基础线路编码'] = df_history_10min['线路编码'].str.rsplit(' - ', n=1).str[0]

# 2.2 聚合历史数据
print("聚合历史 10 分钟数据到每日总量...")
df_actual_daily = df_history_10min.groupby(['线路编码', pd.Grouper(key='日期', freq='D')])['包裹量'].sum().reset_index()
df_actual_daily.rename(columns={'包裹量': '总实际包裹量'}, inplace=True)
# 2.3 合并数据
print("合并实际、预知和线路信息...")
df_merged = pd.merge(df_actual_daily, df_known_forecast, on=['线路编码', '日期'], how='left')
df_merged = pd.merge(df_merged, df_routes_info[['线路编码', '发运节点']], on='线路编码',
how='left')
df_merged['包裹量'].fillna(0, inplace=True)
df_merged.rename(columns={'包裹量': '预知包裹量'}, inplace=True)
# 2.4 特征工程
print("进行特征工程...")
df_merged['星期几'] = df_merged['日期'].dt.dayofweek
df_merged = df_merged.sort_values(by=['线路编码', '日期'])
lags = [1, 2, 3, 7]

```

```

for lag in lags: df_merged[f'实际总量_lag_{lag}'] = df_merged.groupby('线路编码')['总实际包裹量'].shift(lag)
df_merged.dropna(inplace=True)
print("数据预处理完成。")

```

--- 3. 模型定义---

```

def build_combined_model(seq_shape, static_shape):
    sequence_input = Input(shape=seq_shape, name='sequence_input'); lstm_layer =
    LSTM(64, activation='relu', return_sequences=True)(sequence_input); lstm_layer =
    Dropout(0.2)(lstm_layer); lstm_layer = LSTM(32, activation='relu')(lstm_layer); lstm_output =
    Dropout(0.2)(lstm_layer)
    static_input = Input(shape=static_shape, name='static_input'); dense_layer = Dense(32,
    activation='relu')(static_input); dense_layer = Dropout(0.2)(dense_layer); dense_output =
    Dense(16, activation='relu')(dense_layer)
    combined = concatenate([lstm_output, dense_output]); final_output = Dense(32,
    activation='relu')(combined); final_output = Dropout(0.2)(final_output); final_output = Dense(1,
    name='output')(final_output)
    model = Model(inputs=[sequence_input, static_input], outputs=final_output);
    model.compile(optimizer=Adam(learning_rate=0.001), loss='mse'); return model

```

--- 4. 并行训练与预测---

```

def process_route(route_code, df_route_data, df_known_forecast_all, seq_len, lags_list):
    # ... (process_route 函数逻辑与上一版本相同) ...
    if len(df_route_data) < seq_len + 1: return route_code, None
    sequence_features = [f'实际总量_lag_{lag}' for lag in lags_list]; static_features = ['预知包裹量', '星期几']; target_feature = '总实际包裹量'
    X_seq_list, X_static_list, y_list = [], [], [];
    for i in range(seq_len, len(df_route_data)): seq_indices = df_route_data.index[i-seq_len : i];
    X_seq_list.append(df_route_data.loc[seq_indices, sequence_features].values); target_index =
    df_route_data.index[i]; X_static_list.append(df_route_data.loc[target_index,
    static_features].values); y_list.append(df_route_data.loc[target_index, target_feature])
    if not X_seq_list: return route_code, None
    X_seq_train = np.array(X_seq_list); X_static_train = np.array(X_static_list); y_train =
    np.array(y_list).reshape(-1, 1)
    scaler_seq, scaler_static, scaler_y = MinMaxScaler(), MinMaxScaler(), MinMaxScaler()
    nsamples, nsteps, nfeatures_seq = X_seq_train.shape; X_seq_train_scaled =
    scaler_seq.fit_transform(X_seq_train.reshape(-1, nfeatures_seq)).reshape(nsamples, nsteps,
    nfeatures_seq); X_static_train_scaled = scaler_static.fit_transform(X_static_train); y_train_scaled
    = scaler_y.fit_transform(y_train)
    try: model = build_combined_model(seq_shape=(X_seq_train_scaled.shape[1],
    X_seq_train_scaled.shape[2]), static_shape=(X_static_train_scaled.shape[1,]));
    model.fit([X_seq_train_scaled, X_static_train_scaled], y_train_scaled, epochs=50, batch_size=4,
    verbose=0)
    except Exception as e: return route_code, None

```

```

last_sequence_indices = df_route_data.index[-seq_len:]; last_sequence_unscaled =
df_route_data.loc[last_sequence_indices, sequence_features].values
known_forecast_pred_day_series = df_known_forecast_all[(df_known_forecast_all['线路编
码'] == route_code) & (df_known_forecast_all['日期'] == PREDICTION_DATE)][['包裹量'];
known_forecast_pred_day = known_forecast_pred_day_series.iloc[0] if not
known_forecast_pred_day_series.empty else 0
day_of_week_pred_day = PREDICTION_DATE.dayofweek; static_pred_day_unscaled =
np.array([[known_forecast_pred_day, day_of_week_pred_day]])
last_sequence_scaled = scaler_seq.transform(last_sequence_unscaled.reshape(-1,
nfeatures_seq)).reshape(1, seq_len, nfeatures_seq); static_pred_day_scaled =
scaler_static.transform(static_pred_day_unscaled)
predicted_scaled = model.predict([last_sequence_scaled, static_pred_day_scaled]);
predicted_total_volume = scaler_y.inverse_transform(predicted_scaled)[0][0]
return route_code, max(0, predicted_total_volume)

```

```

print("开始模型训练和预测 (并行)...")
all_route_codes_in_merged = df_merged['线路编码'].unique() # 使用 merged 中的线路进行训
练
results = Parallel(n_jobs=N_JOBS, backend='loky')(
    delayed(process_route)(route_code, df_merged[df_merged['线路编码'] ==
route_code].copy(), df_known_forecast, SEQUENCE_LENGTH, lags)
    for route_code in tqdm(all_route_codes_in_merged, desc="训练/预测进度")
)
predictions_total = {result[0]: result[1] for result in results if result and result[1] is not None}
print(f"模型训练和预测完成。成功预测 {len(predictions_total)} 条线路。")

```

--- 5. 拆分到 10 分钟粒度 ---

5.1 计算节点历史比例

```

print("计算节点历史比例 (基于附件 2 数据)...")
historical_ratio_0600 = {}
# 按基础线路编码分组处理历史数据
base_route_groups = df_history_10min.groupby('基础线路编码')
for base_route, group in tqdm(base_route_groups, desc="计算历史比例"):
    daily_ratios = []
    # 按天计算比例
    for date, day_group in group.groupby(pd.Grouper(key='日期', freq='D')):
        prev_date = date - pd.Timedelta(days=1)
        # 计算 D 日 0600 窗口的量
        vol_prev_night = group[(group['日期'] == prev_date) & (group['分钟起始'] >=
time_2100)][['包裹量']].sum()
        vol_curr_morning = day_group[day_group['分钟起始'] < time_0600][['包裹量']].sum()
        vol_0600_d = vol_prev_night + vol_curr_morning
        # 计算 D 日 1400 窗口的量

```

```

        vol_1400_d = day_group[ (day_group['分钟起始'] >= time_1100) & (day_group['分钟
起始'] < time_1400) ]['包裹量'].sum()
        # 计算当日总相关量和比例
        total_relevant_vol_d = vol_0600_d + vol_1400_d
        if total_relevant_vol_d > 1e-6: # 避免除以零
            ratio_0600_d = vol_0600_d / total_relevant_vol_d
            daily_ratios.append(ratio_0600_d)
        # 计算该基础线路的平均比例
        if daily_ratios: # 如果至少有一天有效数据
            avg_ratio = np.mean(daily_ratios)
        else: # 无有效历史数据计算比例
            # print(f"警告：基础线路 {base_route} 无有效历史数据计算节点比例，将使用默认
            值 0.5。")
            avg_ratio = 0.5 # 默认值
        # 将计算出的平均比例赋给所有以此开头的具体线路编码
        # (先暂存，因为 predictions_total 可能不包含所有历史线路)
        # 我们在后面应用时查找基础线路对应的比例
        historical_ratio_0600[base_route] = avg_ratio

print("节点历史比例计算完成。")

```

5.2 计算特定窗口的 Profile (保持不变，使用修正后的鲁棒性逻辑)

```

print("计算特定窗口的历史到达分布 (增加鲁棒性)...")
profiles_0600 = {}
profiles_1400 = {}
# ... (时间点映射逻辑不变) ...
time_slots_str = [f"{h:02d}:{m:02d}:00" for h in range(24) for m in range(0, 60, 10)];
time_slots_time = [pd.to_datetime(t, format="%H:%M:%S").time() for t in time_slots_str]
time_slots_0600_profile = [t for t in time_slots_time if (t >= time_2100) or (t < time_0600)];
map_0600_profile = {time_val: i for i, time_val in enumerate(time_slots_0600_profile)};
num_slots_0600 = len(time_slots_0600_profile)
time_slots_1400_profile = [t for t in time_slots_time if (t >= time_1100) and (t < time_1400)];
map_1400_profile = {time_val: i for i, time_val in enumerate(time_slots_1400_profile)};
num_slots_1400 = len(time_slots_1400_profile)

# 计算 Profile
grouped_history_for_profile = df_history_10min.groupby(['线路编码'])
for route_code in tqdm(predictions_total.keys(), desc="计算 Profile 进度"): # 仅为预测的线路计
    算 Profile
        profile_0600 = np.zeros(num_slots_0600); profile_1400 = np.zeros(num_slots_1400)
        total_vol_window_0600 = 0; total_vol_window_1400 = 0
        try: route_history = grouped_history_for_profile.get_group(route_code)
        except KeyError: profiles_0600[route_code] = np.ones(num_slots_0600)/num_slots_0600;

```



```

profiles_1400[route_code] = np.ones(num_slots_1400)/num_slots_1400; continue
    for date, group in route_history.groupby(pd.Grouper(key='日期', freq='D')):
        # ... (计算窗口内包裹量并累加到 profile 和 total_vol_window 的逻辑不变) ...
        prev_date = date - pd.Timedelta(days=1); vol_prev_night =
route_history[(route_history['日期'] == prev_date) & (route_history['分钟起始'] >= time_2100)];
vol_curr_morning = route_history[(route_history['日期'] == date) & (route_history['分钟起始'] <
time_0600)]; window_0600_data = pd.concat([vol_prev_night, vol_curr_morning])
        window_0600_total_vol_day = window_0600_data['包裹量'].sum();
total_vol_window_0600 += window_0600_total_vol_day
        if window_0600_total_vol_day > 0:
            for _, row in window_0600_data.iterrows():
                if row['分钟起始'] in map_0600_profile:
profile_0600[map_0600_profile[row['分钟起始']]] += row['包裹量']
            window_1400_data = group[(group['分钟起始'] >= time_1100) & (group['分钟起始'] <
time_1400)]
            window_1400_total_vol_day = window_1400_data['包裹量'].sum();
total_vol_window_1400 += window_1400_total_vol_day
            if window_1400_total_vol_day > 0:
                for _, row in window_1400_data.iterrows():
                    if row['分钟起始'] in map_1400_profile:
profile_1400[map_1400_profile[row['分钟起始']]] += row['包裹量']
        # 归一化
        profiles_0600[route_code] = profile_0600/total_vol_window_0600 if
total_vol_window_0600 > 1e-6 else np.ones(num_slots_0600)/num_slots_0600
        profiles_1400[route_code] = profile_1400/total_vol_window_1400 if
total_vol_window_1400 > 1e-6 else np.ones(num_slots_1400)/num_slots_1400

```

5.3 应用 Profile 生成结果

```

print("应用分布生成 10 分钟预测...")
output_timestamps = pd.date_range(start=OUTPUT_START_TIME, end=OUTPUT_END_TIME,
freq='10min')
results_list_10min = []
def get_dispatch_node(route_code):
    if "0600" in route_code:
        return time_0600
    elif "1400" in route_code:
        return time_1400
    else:
        return None
for route_code, predicted_total in tqdm(predictions_total.items(), desc="生成 10 分钟预测"):
    node_time = get_dispatch_node(route_code) # 判断当前线路类型

    # 获取对应的 Profile (如果不存在则跳过)
    if node_time == time_0600:

```

```

        if route_code not in profiles_0600: continue
        profile_to_use = profiles_0600[route_code]
    elif node_time == time_1400:
        if route_code not in profiles_1400: continue
        profile_to_use = profiles_1400[route_code]
    else:
        # print(f'警告：线路 {route_code} 无法确定节点类型，跳过拆分。")
        continue # 对于无法确定节点类型的线路，不进行拆分

    for ts in output_timestamps:
        current_time = ts.time()
        predicted_volume_at_ts = 0 # 默认值为 0

        # --- 根据线路类型和时间戳，仅在对应窗口内计算 ---
        # 如果是 0600 线路，只在其窗口 [D-1 21:00, D 06:00) 内计算
        if node_time == time_0600 and \
            ((ts.date() == PREDICTION_DATE.date() and current_time < time_0600) or \
             (ts.date() == (PREDICTION_DATE - pd.Timedelta(days=1)).date() and
              current_time >= time_2100)):
            if current_time in map_0600_profile: # 确保时间点在映射中
                profile_index = map_0600_profile[current_time]
                # 直接用总预测量乘以对应 profile 值
                predicted_volume_at_ts = predicted_total * profile_to_use[profile_index]

            # 如果是 1400 线路，只在其窗口 [D 11:00, D 14:00) 内计算
            elif node_time == time_1400 and \
                (ts.date() == PREDICTION_DATE.date() and current_time >= time_1100 and
                 current_time < time_1400):
                if current_time in map_1400_profile: # 确保时间点在映射中
                    profile_index = map_1400_profile[current_time]
                    # 直接用总预测量乘以对应 profile 值
                    predicted_volume_at_ts = predicted_total * profile_to_use[profile_index]

            # 其他所有情况，预测量保持为 0

        results_list_10min.append({'线路编码': route_code, '时间': ts, '预测包裹量': max(0,
round(predicted_volume_at_ts))})

df_result_10min = pd.DataFrame(results_list_10min)
print("拆分完成。")

# --- 6. 可视化指定线路结果---
print("开始生成美化版可视化图表...")

```

```

os.makedirs(OUTPUT_DIR, exist_ok=True)

# 设置 Seaborn 风格
sns.set_theme(style="whitegrid", palette="muted")
sns.set(font='Microsoft YaHei') # 解决 Seaborn 中文显示问题

for route_code in TARGET_ROUTES_FOR_VIS:
    if route_code in predictions_total:
        print(f"\n--- 线路: {route_code} 预测详情 ---")
        predicted_total_volume = predictions_total[route_code]
        print(f" 预测总包裹量 (12 月 16 日): {predicted_total_volume:.2f}")

        df_vis = df_result_10min[df_result_10min['线路编码'] == route_code].copy()
        if not df_vis.empty:
            # 确定节点类型
            node_time = get_dispatch_node(route_code) # 需要 get_dispatch_node 函数

            plt.figure(figsize=(18, 7))

            # 绘制预测曲线
            sns.lineplot(data=df_vis, x='时间', y='预测包裹量',
                        marker='o', markersize=4,
                        label=f'预测包裹量 (总量: {predicted_total_volume:.0f})',
                        linewidth=1.5)

            # --- 添加累积窗口高亮区域 ---
            highlight_color = 'lightgrey'; alpha_level = 0.4
            if node_time == time_0600:
                start_highlight = pd.Timestamp.combine(PREDICTION_DATE.date() -
pd.Timedelta(days=1), time_2100)
                end_highlight = pd.Timestamp.combine(PREDICTION_DATE.date(),
time_0600)
                window_label = "06:00 节点累积窗口 (前一天 21:00 - 当天 06:00)"
            elif node_time == time_1400:
                start_highlight = pd.Timestamp.combine(PREDICTION_DATE.date(),
time_1100)
                end_highlight = pd.Timestamp.combine(PREDICTION_DATE.date(),
time_1400)
                window_label = "14:00 节点累积窗口 (当天 11:00 - 当天 14:00)"
            else:
                start_highlight, end_highlight, window_label = None, None, "未知节点类型"

            if start_highlight and end_highlight:
                plt.axvspan(start_highlight, end_highlight, color=highlight_color,

```

```
alpha=alpha_level, label=window_label)
```

```
    # --- 优化坐标轴和标题 ---
    ax = plt.gca()
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d %H:%M'))
    ax.xaxis.set_major_locator(mdates.HourLocator(interval=2))
    ax.xaxis.set_minor_locator(mdates.HourLocator(interval=1))
    plt.gcf().autofmt_xdate(rotation=30)
    max_y = df_vis['预测包裹量'].max()
    # 增加判断防止 max_y 为 0 或 NaN 时出错
    if pd.isna(max_y) or max_y <= 0: max_y = 10 # 如果最大值为 0 或无效, 给一个
默认上限
    ax.set_ylim(bottom=-max_y*0.05, top=max_y * 1.15 if max_y > 0 else 10)
    ax.set_ylabel("预测包裹量 (10 分钟粒度)", fontsize=12) # 使用配置好的字体
    ax.set_xlabel("时间", fontsize=12) # 使用配置好的字体
    title_str = f'线路 {route_code} 的 10 分钟粒度预测包裹量\n(时间窗口:
{OUTPUT_START_TIME.strftime("%Y-%m-%d %H:%M")} to
{OUTPUT_END_TIME.strftime("%Y-%m-%d %H:%M")})'
    plt.title(title_str, fontsize=14, fontweight='bold') # 使用配置好的字体

    # 图例
    handles, labels = ax.get_legend_handles_labels()
    order = [i for i, label in enumerate(labels) if "窗口" not in label] + \
            [i for i, label in enumerate(labels) if "窗口" in label]
    # 使用 prop 参数指定字体属性 (如果全局设置无效, 可以尝试这种方式)
    # font_prop = fm.FontProperties(fname=font_path) if font_path else None
    # ax.legend([handles[idx] for idx in order], [labels[idx] for idx in order], loc='upper
left', prop=font_prop)
    ax.legend([handles[idx] for idx in order], [labels[idx] for idx in order], loc='upper
left')

    plt.tight_layout(pad=1.5)
    vis_filename = os.path.join(OUTPUT_DIR, f'{route_code.replace(' ', '_')}_prediction_vis_enhanced.png')
    plt.savefig(vis_filename, dpi=150)
    print(f" 美化版可视化图表已保存至: {vis_filename}")
    plt.close()
else:
    print(f" 线路 {route_code} 无 10 分钟预测结果可供可视化。")
else:
    print(f"\n 线路: {route_code} 未能成功预测或未在预测结果中。")

print("可视化完成。")
```

--- 7. 更新结果表 (保持不变) ---

print("开始更新结果表文件...")

7.1 更新结果表 1

try:

```
    print(f'更新 {RESULT_TABLE1_PATH}...')
    df_pred_total_new = pd.DataFrame(list(predictions_total.items()), columns=['线路编码', '预测货量']); df_pred_total_new['预测货量'] = df_pred_total_new['预测货量'].round().astype(int);
    df_pred_total_new['日期'] = PREDICTION_DATE
    df_output1 = df_output1.set_index(['线路编码', '日期']); df_pred_total_new = df_pred_total_new.set_index(['线路编码', '日期'])
    df_output1['货量'].update(df_pred_total_new['预测货量']);
    df_output1.reset_index(inplace=True); df_output1['货量'] = pd.to_numeric(df_output1['货量'], errors='coerce').fillna(0).astype(int)
    df_output1.to_excel(RESULT_TABLE1_PATH, index=False, engine='openpyxl');
    print(f'{RESULT_TABLE1_PATH} 更新成功。')
    print(f'提示：如果在 Excel 中看到 '#####', 请手动调宽列。")
```

except Exception as e:

```
    print(f'更新 {RESULT_TABLE1_PATH} 失败：{e}'); backup_filename = os.path.join(OUTPUT_DIR, "Result_Table1_Total_Prediction_Backup.xlsx")
    df_pred_total_new.reset_index().to_excel(backup_filename, index=False); print(f'预测结果已备份至：{backup_filename}')
```

7.2 更新结果表 2

try:

```
    print(f'更新 {RESULT_TABLE2_PATH}...')
    if not df_result_10min.empty:
        df_pred_10min_new = df_result_10min.copy(); df_pred_10min_new['日期'] = df_pred_10min_new['时间'].dt.date; df_pred_10min_new['分钟起始'] = df_pred_10min_new['时间'].dt.time
        df_pred_10min_new.rename(columns={'预测包裹量': '包裹量'}, inplace=True);
        df_pred_10min_new = df_pred_10min_new[['线路编码', '日期', '分钟起始', '包裹量']]
        df_pred_10min_new['日期'] = pd.to_datetime(df_pred_10min_new['日期'])
        df_output2 = df_output2.set_index(['线路编码', '日期', '分钟起始']);
        df_pred_10min_new = df_pred_10min_new.set_index(['线路编码', '日期', '分钟起始'])
        df_output2['包裹量'].update(df_pred_10min_new['包裹量']);
        df_output2.reset_index(inplace=True); df_output2['包裹量'] = pd.to_numeric(df_output2['包裹量'], errors='coerce').fillna(0).astype(int)
        df_output2['分钟起始'] = df_output2['分钟起始'].astype(str) # 保存为字符串
        df_output2.to_excel(RESULT_TABLE2_PATH, index=False, engine='openpyxl');
    print(f'{RESULT_TABLE2_PATH} 更新成功。')
    print(f'提示：如果在 Excel 中看到 '#####', 请手动调宽列。")
```

else: print("无 10 分钟预测结果可用于更新结果表 2。")

except Exception as e:

```

print(f'更新 {RESULT_TABLE2_PATH} 失败: {e}')
if not df_result_10min.empty: backup_filename = os.path.join(OUTPUT_DIR,
"Result_Table2_10min_Prediction_Backup.xlsx"); df_pred_10min_new_save =
df_pred_10min_new.reset_index(); df_pred_10min_new_save['分钟起始'] =
df_pred_10min_new_save['分钟起始'].astype(str);
df_pred_10min_new_save.to_excel(backup_filename, index=False); print(f'10 分钟预测结果已
备份至: {backup_filename}')

```

--- 结束计时 ---

```
end_time = time.time()
```

```
total_time = end_time - start_time
```

```
print(f'\n 处理完成！总耗时: {total_time:.2f} 秒')
```

11.2 第二题主代码程序：

```

import pandas as pd
import numpy as np
from tqdm import tqdm
import os
import warnings
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import time
from datetime import datetime, timedelta
from itertools import combinations # 用于生成组合
# 导入 OR-Tools CP-SAT
from ortools.sat.python import cp_model

warnings.filterwarnings('ignore')
# 设置 Matplotlib
try: plt.rcParams['font.sans-serif'] = ['SimHei']
except:
    try: plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
    except: print("警告：未找到 SimHei 或 Microsoft YaHei 字体，中文可能显示为方块。")
plt.rcParams['axes.unicode_minus'] = False

# --- 0. 参数与常量 ---
PREDICTION_DATE = pd.to_datetime("2024-12-16")
OUTPUT_START_TIME = pd.to_datetime("2024-12-15 14:00:00")
OUTPUT_END_TIME = pd.to_datetime("2024-12-16 14:00:00")
TIME_HORIZON_START_DT = pd.to_datetime("2024-12-15 00:00:00")
RESULT_TABLE2_PATH = '结果表 2.xlsx'
RESULT_TABLE3_PATH = '结果表 3.xlsx'
FLEET_INFO_PATH = '附件 5.xlsx'
ROUTE_INFO_PATH = '附件 1.xlsx'
OUTPUT_DIR = "./results"

```

```

VEHICLE_CAPACITY = 1000
LOADING_TIME_MIN = 45
UNLOADING_TIME_MIN = 45
VEHICLE_FIXED_COST = 100
EXTERNAL_COST_MULTIPLIER = 2.0
MAX_STOPS_CONSOLIDATION = 3 # 串点最大站点数

W_COST = 1.0; W_TURNOVER = 0.5; W_FILLRATE = 0.2
time_2100_dt = pd.to_datetime("21:00:00", format="%H:%M:%S").time(); time_0600_dt =
pd.to_datetime("06:00:00", format="%H:%M:%S").time()
time_1100_dt = pd.to_datetime("11:00:00", format="%H:%M:%S").time(); time_1400_dt =
pd.to_datetime("14:00:00", format="%H:%M:%S").time()
node_time_0600 = pd.Timestamp.combine(PREDICTION_DATE.date(), time_0600_dt);
node_time_1400 = pd.Timestamp.combine(PREDICTION_DATE.date(), time_1400_dt)

# Helper functions
def time_to_minutes(dt): return int((dt - TIME_HORIZON_START_DT).total_seconds() / 60)
def minutes_to_time(mins): return TIME_HORIZON_START_DT + timedelta(minutes=mins)

# --- 1. 加载所有需要的数据 ---
start_time = time.time()
print("加载数据...")

try:
    df_pred_10min = pd.read_excel(RESULT_TABLE2_PATH); df_routes_info =
pd.read_excel(ROUTE_INFO_PATH); df_fleet_info = pd.read_excel(FLEET_INFO_PATH);
df_output3 = pd.read_excel(RESULT_TABLE3_PATH)
except FileNotFoundError as e: print(f"错误: 找不到所需文件 - {e}。"); exit()
print("数据加载完成。")

# --- 2. 数据预处理 ---
print("数据清洗与预处理...")

df_pred_10min['日期'] = pd.to_datetime(df_pred_10min['日期'])
try: df_pred_10min['分钟起始'] = pd.to_datetime(df_pred_10min['分钟起始'],
format="%H:%M:%S").dt.time
except: df_pred_10min['分钟起始'] = pd.to_datetime(df_pred_10min['分钟起始'].astype(str),
errors='coerce').dt.time
df_pred_10min['包裹量'] = pd.to_numeric(df_pred_10min['包裹量'], errors='coerce').fillna(0)
df_pred_10min['时间'] = df_pred_10min.apply(lambda row: pd.Timestamp.combine(row['日期']
.date(), row['分钟起始']) if pd.notna(row['日期']) and pd.notna(row['分钟起始']) else pd.NaT,
axis=1)
df_pred_10min = df_pred_10min.dropna(subset=['时间'])

```

```

df_routes_info['线路编码'] = df_routes_info['线路编码'].str.strip(); df_fleet_info['车队编码'] =
df_fleet_info['车队编码'].str.strip(); df_output3['日期'] = pd.to_datetime(df_output3['日期'])
df_routes_info['发运节点'] = pd.to_datetime(df_routes_info['发运节点'],
format='%H:%M:%S').dt.time; df_pred_10min['线路编码'] = df_pred_10min['线路编码']
].str.strip()
df_routes_merged = pd.merge(df_routes_info, df_fleet_info, on='车队编码', how='left');
route_details = df_routes_merged.set_index('线路编码').T.to_dict()
print("数据加载和预处理完成。")

```

--- 步骤一：生成潜在任务（仅满载和尾货候选）---

```
print("生成潜在运输任务（满载和尾货候选）...")
```

```
potential_full_tasks = [] # 只存满载任务
```

```
leftover_candidates = [] # 存所有尾货信息
```

```
all_routes = df_pred_10min['线路编码'].unique()
```

```
for route_code in tqdm(all_routes, desc="处理线路生成任务"):
```

```
    route_info = route_details.get(route_code);
```

```
    if not route_info: continue
```

```
    node_time = route_info['发运节点']; cumulative_volume = 0; full_loads_dispatched = 0
```

```
    if node_time == time_0600_dt: node_time_dt = node_time_0600; window_start_dt =
pd.Timestamp.combine(PREDICTION_DATE.date() - timedelta(days=1), time_2100_dt);
window_end_dt = node_time_dt
```

```
    elif node_time == time_1400_dt: node_time_dt = node_time_1400; window_start_dt =
pd.Timestamp.combine(PREDICTION_DATE.date(), time_1100_dt); window_end_dt =
node_time_dt
```

```
    else: continue
```

```
    df_route_window = df_pred_10min[(df_pred_10min['线路编码'] == route_code) &
(df_pred_10min['时间'] >= window_start_dt) & (df_pred_10min['时间'] <
window_end_dt)].sort_values(by='时间')
```

```
    for index, row in df_route_window.iterrows():
```

```
        current_time_dt = row['时间']; cumulative_volume += row['包裹量']
```

```
        while cumulative_volume >= VEHICLE_CAPACITY:
```

```
            task_id = f"{route_code}_Full_{full_loads_dispatched + 1}"; earliest_dispatch =
current_time_dt; latest_dispatch = node_time_dt
```

```
            potential_full_tasks.append({'task_id': task_id, 'type': 'Full', 'routes': [route_code],
'volume': VEHICLE_CAPACITY, 'earliest_dispatch_dt': earliest_dispatch, 'latest_dispatch_dt':
latest_dispatch, 'node_time_dt': node_time_dt, 'required_fleet': route_info['车队编码'],
'travel_time_hr': route_info['在途时长'], 'variable_cost': route_info['自有变动成本']})
```

```
            cumulative_volume -= VEHICLE_CAPACITY; full_loads_dispatched += 1
```

```
        if cumulative_volume > 1e-6: # 只有明确大于 0 的才算尾货
```

```
            leftover_candidates.append({'original_index': len(leftover_candidates), # 添加原始索引方便查找
```



```

                                'route_code':      route_code,      'volume':
cumulative_volume, 'node_time_dt': node_time_dt,
                                'required_fleet':  route_info[' 车 队 编 码 '],
'travel_time_hr': route_info['在途时长'],
                                'variable_cost':  route_info[' 自 有 变 动 成 本 '],
'base_route': route_code.rsplit(' - ', 1)[0])

```

--- 步骤二: 最优串点 (使用 CP-SAT Set Covering) ---

```
print("执行最优串点 (CP-SAT Set Covering)...")
```

```
consolidated_tasks = []
```

```
unconsolidated_leftovers = []
```

```
grouped_leftovers = {}
```

按场地和节点时间分组

```
for item in leftover_candidates:
```

```
    key = (item['base_route'].split(' - ')[0], item['node_time_dt'])
```

```
    grouped_leftovers.setdefault(key, []).append(item)
```

MAX_GROUP_SIZE_FOR_OPTIMAL = 50 # 设置一个阈值, 过大的组使用启发式或报错

```
for key, items in tqdm(grouped_leftovers.items(), desc="处理串点组"):
```

```
    num_items = len(items)
```

```
    if num_items == 0: continue
```

```
    if num_items == 1: # 只有一个尾货, 直接加入单独列表
```

```
        item = items[0]
```

```
        task_id = f'{item["route_code"]}_Leftover'
```

```
        unconsolidated_leftovers.append({'task_id': task_id, 'type': 'Leftover', 'routes':
[item['route_code']], 'volume': item['volume'], 'earliest_dispatch_dt': item['node_time_dt'],
'latest_dispatch_dt': item['node_time_dt'], 'node_time_dt': item['node_time_dt'], 'required_fleet':
item['required_fleet'], 'travel_time_hr': item['travel_time_hr'], 'variable_cost':
item['variable_cost']})
```

```
        continue
```

如果组内物品过多, 可能导致组合爆炸, 可以考虑使用启发式或报错

```
if num_items > MAX_GROUP_SIZE_FOR_OPTIMAL:
```

```
    print(f"警告: 组 {key} 的尾货数量 ({num_items}) 过多, 超过阈值
{MAX_GROUP_SIZE_FOR_OPTIMAL}。将对该组使用简化的单独发运策略。")
```

```
    # 简化策略: 该组所有尾货都单独发运
```

```
    for item in items:
```

```
        task_id = f'{item["route_code"]}_Leftover_LargeGroup'
```

```
        unconsolidated_leftovers.append({'task_id': task_id, 'type': 'Leftover', 'routes':
[item['route_code']], 'volume': item['volume'], 'earliest_dispatch_dt': item['node_time_dt'],
'latest_dispatch_dt': item['node_time_dt'], 'node_time_dt': item['node_time_dt'], 'required_fleet':
item['required_fleet'], 'travel_time_hr': item['travel_time_hr'], 'variable_cost':
item['variable_cost']})
```

```

        continue # 处理下一个组

# --- 构建 Set Covering 模型 ---
model = cp_model.CpModel()
item_indices = range(num_items)

# 1. 生成所有有效的串点集合 (2 或 3 个)
potential_sets = [] # [(set_id, [item_idx1, item_idx2, ...], total_volume), ...]
set_counter = 0
for combo_size in range(2, MAX_STOPS_CONSOLIDATION + 1):
    for combo_indices in combinations(item_indices, combo_size):
        combo_items = [items[i] for i in combo_indices]
        total_volume = sum(item['volume'] for item in combo_items)
        if total_volume <= VEHICLE_CAPACITY:
            potential_sets.append((set_counter, list(combo_indices), total_volume))
            set_counter += 1

# 2. 创建决策变量
# set_vars[s_idx]: 是否选择串点集合 s
set_vars = [model.NewBoolVar(f'set_{s[0]}') for s in potential_sets]
# single_vars[i_idx]: 是否单独运输物品 i
single_vars = [model.NewBoolVar(f'single_{i}') for i in item_indices]

# 3. 添加覆盖约束: 每个物品必须被覆盖
# item_to_sets[i]: 包含物品 i 的所有 potential_sets 的索引列表
item_to_sets = {i: [] for i in item_indices}
for s_idx, s_data in enumerate(potential_sets):
    for item_idx in s_data[1]:
        item_to_sets[item_idx].append(s_idx)

for i in item_indices:
    model.Add(sum(set_vars[s_idx] for s_idx in item_to_sets[i]) + single_vars[i] == 1)

# 4. 设置目标函数: 最小化总车辆数
model.Minimize(sum(set_vars) + sum(single_vars))

# 5. 求解模型
solver = cp_model.CpSolver()
# solver.parameters.log_search_progress = False # 可以关闭日志减少输出
status = solver.Solve(model)

# 6. 解析结果
if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:

```

```

# 提取被选中的串点集合
for s_idx, s_data in enumerate(potential_sets):
    if solver.Value(set_vars[s_idx]) == 1:
        combo_indices = s_data[1]
        combo_items = [items[i] for i in combo_indices]
        task_id = f"Consolidated_{key[0]}_{key[1].strftime('%H%M')}_{s_idx}"
        routes = [item['route_code'] for item in combo_items]
        max_travel_time = max(item['travel_time_hr'] for item in combo_items)
        max_var_cost = max(item['variable_cost'] for item in combo_items)
        consolidated_tasks.append({'task_id': task_id, 'type': 'Consolidated', 'routes':
routes, 'volume': s_data[2], 'earliest_dispatch_dt': key[1], 'latest_dispatch_dt': key[1],
'node_time_dt': key[1], 'required_fleet': combo_items[0]['required_fleet'], 'travel_time_hr':
max_travel_time, 'variable_cost': max_var_cost})
    # 提取单独运输的物品
    for i in item_indices:
        if solver.Value(single_vars[i]) == 1:
            item = items[i]
            task_id = f"{item['route_code']}_Leftover"
            unconsolidated_leftovers.append({'task_id': task_id, 'type': 'Leftover', 'routes':
[item['route_code']], 'volume': item['volume'], 'earliest_dispatch_dt': item['node_time_dt'],
'latest_dispatch_dt': item['node_time_dt'], 'node_time_dt': item['node_time_dt'], 'required_fleet':
item['required_fleet'], 'travel_time_hr': item['travel_time_hr'], 'variable_cost':
item['variable_cost']})
        else:
            print(f"警告：组 {key} 的最优串点求解失败 (状态: {solver.StatusName(status)}).
将对该组使用简化的单独发运策略。")
            # 求解失败，则该组所有尾货都单独发运
            for item in items:
                task_id = f"{item['route_code']}_Leftover_SolveFail"
                unconsolidated_leftovers.append({'task_id': task_id, 'type': 'Leftover', 'routes':
[item['route_code']], 'volume': item['volume'], 'earliest_dispatch_dt': item['node_time_dt'],
'latest_dispatch_dt': item['node_time_dt'], 'node_time_dt': item['node_time_dt'], 'required_fleet':
item['required_fleet'], 'travel_time_hr': item['travel_time_hr'], 'variable_cost':
item['variable_cost']})

# 合并所有最终确定的任务
final_tasks = potential_full_tasks + consolidated_tasks + unconsolidated_leftovers
print(f"任务生成与最优串点完成，共 {len(final_tasks)} 个运输任务。")

# --- 步骤三：构建并求解 CP-SAT 调度模型 ---
# ... (CP-SAT 调度模型代码与上一个版本完全相同，处理 final_tasks 列表) ...
print("构建并求解 CP-SAT 调度模型...")

```

```

model_sched = cp_model.CpModel()
# --- 数据准备 for CP-SAT---
num_tasks = len(final_tasks); task_indices = range(num_tasks)
own_vehicles = []; vehicle_id_counter = 0
for idx, row in df_fleet_info.iterrows(): # 使用 iterrows 获取索引和行
    for i in range(row['自有车数量']): own_vehicles.append({'id': f'Own_{row['车队编码']}_{i+1}', 'fleet': row['车队编码'], 'is_external': False}); vehicle_id_counter += 1
num_own_vehicles = len(own_vehicles); external_vehicles = [{'id': 'External', 'fleet': 'External', 'is_external': True}]; all_vehicles = own_vehicles + external_vehicles; num_all_vehicles = len(all_vehicles); vehicle_indices = range(num_all_vehicles)
horizon_end_min = time_to_minutes(OUTPUT_END_TIME + timedelta(hours=12)) # Extend horizon
task_data = []
for i, task in enumerate(final_tasks):
    earliest_start_min = time_to_minutes(task['earliest_dispatch_dt']); latest_start_min = time_to_minutes(task['latest_dispatch_dt'])
    if latest_start_min < earliest_start_min: latest_start_min = earliest_start_min
    travel_time_min = int(task['travel_time_hr'] * 60)
    duration_min = LOADING_TIME_MIN + travel_time_min
    round_trip_duration_min = LOADING_TIME_MIN + travel_time_min * 2 + UNLOADING_TIME_MIN
    task_data.append({'id': i, 'task_obj': task, 'earliest_start': earliest_start_min, 'latest_start': latest_start_min, 'duration': duration_min, 'round_trip_duration': round_trip_duration_min, 'volume': task['volume'], 'required_fleet': task['required_fleet'], 'variable_cost': task['variable_cost'], 'external_cost': task['variable_cost'] * EXTERNAL_COST_MULTIPLIER})
# --- 创建 CP-SAT 变量 ---
performed_sched = [model_sched.NewBoolVar(f'performed_{i}') for i in task_indices];
[model_sched.Add(performed_sched[i] == 1) for i in task_indices] # All tasks must be performed
start_vars = [model_sched.NewIntVar(task_data[i]['earliest_start'], task_data[i]['latest_start'], f'start_{i}') for i in task_indices]
end_vars = [model_sched.NewIntVar(0, horizon_end_min, f'end_{i}') for i in task_indices]
assignment_vars = {} # (task_id, vehicle_id) -> BoolVar
for i in task_indices:
    for v_idx in vehicle_indices:
        assignment_vars[i, v_idx] = model_sched.NewBoolVar(f'assign_{i}_{all_vehicles[v_idx]["id"]}')
vehicle_used_vars = [model_sched.NewBoolVar(f'used_{v["id"]}') for v in own_vehicles]
interval_vars = {} # vehicle_idx -> list of IntervalVar
# --- 添加 CP-SAT 约束 ---
print("添加调度约束...")
for i in task_indices: model_sched.Add(sum(assignment_vars[i, v_idx] for v_idx in vehicle_indices) == performed_sched[i])
for i in task_indices: model_sched.Add(end_vars[i] == start_vars[i] + task_data[i]['duration'])
for v_idx, vehicle in enumerate(own_vehicles):
    intervals = []

```

```

    for i in task_indices: intervals.append(model_sched.NewOptionalIntervalVar(start_vars[i],
task_data[i]['duration'], end_vars[i], assignment_vars[i, v_idx], f'interval_{i}_{vehicle["id"]}')
    model_sched.AddNoOverlap(intervals)
    interval_vars[v_idx] = intervals
for i in task_indices:
    required_fleet = task_data[i]['required_fleet']
    for v_idx, vehicle in enumerate(all_vehicles):
        if not vehicle['is_external'] and vehicle['fleet'] != required_fleet:
model_sched.Add(assignment_vars[i, v_idx] == 0)
for v_idx, vehicle in enumerate(own_vehicles):
    model_sched.Add(sum(assignment_vars[i, v_idx] for i in task_indices) >
0).OnlyEnforceIf(vehicle_used_vars[v_idx])
    model_sched.Add(sum(assignment_vars[i, v_idx] for i in task_indices) ==
0).OnlyEnforceIf(vehicle_used_vars[v_idx].Not())
# --- 定义目标函数 ---
print("定义调度目标函数...")
total_fixed_cost = sum(vehicle_used_vars[v_idx] * VEHICLE_FIXED_COST for v_idx in
range(num_own_vehicles))
total_variable_cost = sum(assignment_vars[i, v_idx] * int(task_data[i]['variable_cost']) for i in
task_indices for v_idx in range(num_own_vehicles))
total_external_cost = sum(assignment_vars[i, num_own_vehicles] *
int(task_data[i]['external_cost']) for i in task_indices)
total_cost = total_fixed_cost + total_variable_cost + total_external_cost
total_own_tasks = sum(assignment_vars[i, v_idx] for i in task_indices for v_idx in
range(num_own_vehicles))
M_turnover = num_tasks
turnover_objective_term = M_turnover * num_own_vehicles - total_own_tasks if
num_own_vehicles > 0 else 0
total_packages = sum(performed_sched[i] * int(task_data[i]['volume']) for i in task_indices)
total_capacity_used = sum(performed_sched[i] * VEHICLE_CAPACITY for i in task_indices)
fill_rate_objective_term = total_capacity_used - total_packages
SCALE_FACTOR = 100
int_w_cost = int(W_COST * SCALE_FACTOR); int_w_turnover = int(W_TURNOVER *
SCALE_FACTOR)
int_w_fillrate = int(W_FILLRATE * SCALE_FACTOR)
model_sched.Minimize(total_cost * int_w_cost + turnover_objective_term * int_w_turnover +
fill_rate_objective_term * int_w_fillrate)

# --- 求解模型 ---
print("开始求解 CP-SAT 调度模型...")
solver_sched = cp_model.CpSolver(); solver_sched.parameters.log_search_progress = True;
solver_sched.parameters.max_time_in_seconds = 180.0 # 可调整求解时间
status_sched = solver_sched.Solve(model_sched)

```

```

# --- 处理结果 ---
print("处理调度求解结果...")
solution_found_sched = (status_sched == cp_model.OPTIMAL or status_sched ==
cp_model.FEASIBLE)
results_list_final = []

if solution_found_sched:
    print(f"求解成功！状态：{solver_sched.StatusName(status_sched)}\n最优目标值：
{solver_sched.ObjectiveValue()}")
    assigned_tasks_count = 0
    for i in task_indices:
        assigned = False
        for v_idx in range(num_own_vehicles):
            if solver_sched.Value(assignment_vars[i, v_idx]) == 1:
                vehicle_id = own_vehicles[v_idx]['id']; start_time_min =
solver_sched.Value(start_vars[i]); dispatch_time_dt = minutes_to_time(start_time_min);
original_routes = ", ".join(task_data[i]['task_obj']['routes'])
                results_list_final.append({'线路编码': original_routes, '日期':
PREDICTION_DATE.date(), '预计发运时间': dispatch_time_dt, '发运车辆': vehicle_id});
                assigned = True; assigned_tasks_count += 1; break
            if not assigned and solver_sched.Value(assignment_vars[i, num_own_vehicles]) == 1:
                vehicle_id = 'External'; start_time_min = solver_sched.Value(start_vars[i]);
dispatch_time_dt = minutes_to_time(start_time_min); original_routes = ",
".join(task_data[i]['task_obj']['routes'])
                results_list_final.append({'线路编码': original_routes, '日期':
PREDICTION_DATE.date(), '预计发运时间': dispatch_time_dt, '发运车辆': vehicle_id});
                assigned_tasks_count += 1
        print(f"共处理和分配了 {assigned_tasks_count} 个运输任务。")
    df_final_schedule = pd.DataFrame(results_list_final); df_final_schedule['日期'] =
pd.to_datetime(df_final_schedule['日期']); df_final_schedule['预计发运时间'] =
pd.to_datetime(df_final_schedule['预计发运时间'])
    try:
        print(f"更新/保存 {RESULT_TABLE3_PATH}...")
        df_output3_updated = df_final_schedule[['线路编码','日期','预计发运时间','发运车辆']]
        df_output3_updated['日期'] = df_output3_updated['日期'].dt.strftime("%Y/%m/%d")
        df_output3_updated['预计发运时间'] = df_output3_updated['预计发运时间
'].dt.strftime("%H:%M:%S")
        df_output3_updated.to_excel(RESULT_TABLE3_PATH, index=False,
engine='openpyxl')
        print(f"{RESULT_TABLE3_PATH} 更新/保存成功。")
    except Exception as e:
        print(f"更新/保存 {RESULT_TABLE3_PATH} 失败：{e}"); backup_filename =
os.path.join(OUTPUT_DIR, "Result_Table3_Schedule_Backup.xlsx");
df_final_schedule.to_excel(backup_filename, index=False); print(f"调度结果已备份至：

```

```

{backup_filename}")
else:
    print("调度模型未能找到可行解或最优解。请检查模型约束或增加求解时间。")

# --- 结束计时 ---
end_time = time.time(); total_time = end_time - start_time
print(f"\n 问题二处理完成！总耗时: {total_time:.2f} 秒")

11.3 第三题主代码程序：

import pandas as pd
import numpy as np
from tqdm import tqdm
import os
import warnings
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import time
from datetime import datetime, timedelta
from itertools import combinations
from ortools.sat.python import cp_model

warnings.filterwarnings('ignore')

try: plt.rcParams['font.sans-serif'] = ['SimHei']
except:
    try: plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']
    except: print("警告：未找到 SimHei 或 Microsoft YaHei 字体，中文可能显示为方块。")
plt.rcParams['axes.unicode_minus'] = False

# --- 0. 参数与常量 (容器时间/容量) ---
PREDICTION_DATE = pd.to_datetime("2024-12-16")
OUTPUT_START_TIME = pd.to_datetime("2024-12-15 14:00:00"); OUTPUT_END_TIME =
pd.to_datetime("2024-12-16 14:00:00")
TIME_HORIZON_START_DT = pd.to_datetime("2024-12-15 00:00:00")
RESULT_TABLE2_PATH = '结果表 2.xlsx'; RESULT_TABLE4_PATH = '结果表 4.xlsx'
FLEET_INFO_PATH = '附件 5.xlsx'; ROUTE_INFO_PATH = '附件 1.xlsx'
OUTPUT_DIR = "./results"

VEHICLE_CAPACITY_NORMAL = 1000; VEHICLE_CAPACITY_CONTAINER = 800
LOADING_TIME_NORMAL_MIN = 45; UNLOADING_TIME_NORMAL_MIN = 45
LOADING_TIME_CONTAINER_MIN = 10; UNLOADING_TIME_CONTAINER_MIN = 10
VEHICLE_FIXED_COST = 100; EXTERNAL_COST_MULTIPLIER = 2.0
MAX_STOPS_CONSOLIDATION = 3

# <<< 调整权重以反映业务优先级 >>>

```

```

W_COST = 1.0          # 成本权重
W_TURNOVER = 0.5      # 周转率权重
W_FILLRATE = 0.2      # 填充率权重

time_2100_dt = pd.to_datetime("21:00:00", format="%H:%M:%S").time(); time_0600_dt =
pd.to_datetime("06:00:00", format="%H:%M:%S").time()
time_1100_dt = pd.to_datetime("11:00:00", format="%H:%M:%S").time(); time_1400_dt =
pd.to_datetime("14:00:00", format="%H:%M:%S").time()
node_time_0600 = pd.Timestamp.combine(PREDICTION_DATE.date(), time_0600_dt);
node_time_1400 = pd.Timestamp.combine(PREDICTION_DATE.date(), time_1400_dt)

# Helper functions
def time_to_minutes(dt): return int((dt - TIME_HORIZON_START_DT).total_seconds() / 60)
def minutes_to_time(mins): return TIME_HORIZON_START_DT + timedelta(minutes=mins)

# --- 1. 加载所有需要的数据 ---
start_time = time.time(); print("加载数据...")
try:
    df_pred_10min = pd.read_excel(RESULT_TABLE2_PATH); df_routes_info =
pd.read_excel(ROUTE_INFO_PATH); df_fleet_info = pd.read_excel(FLEET_INFO_PATH);
df_output4 = pd.read_excel(RESULT_TABLE4_PATH)
except FileNotFoundError as e: print(f"错误: 找不到所需文件 - {e}。"); exit()
print("数据加载完成。")

# --- 2. 数据预处理 ---
print("数据清洗与预处理...")

df_pred_10min['日期'] = pd.to_datetime(df_pred_10min['日期'])
try: df_pred_10min['分钟起始'] = pd.to_datetime(df_pred_10min['分钟起始'],
format="%H:%M:%S").dt.time
except: df_pred_10min['分钟起始'] = pd.to_datetime(df_pred_10min['分钟起始'].astype(str),
errors='coerce').dt.time
df_pred_10min['包裹量'] = pd.to_numeric(df_pred_10min['包裹量'], errors='coerce').fillna(0)
df_pred_10min['时间'] = df_pred_10min.apply(lambda row: pd.Timestamp.combine(row['日期']
).date(), row['分钟起始']) if pd.isna(row['日期']) and pd.isna(row['分钟起始']) else pd.NaT,
axis=1)
df_pred_10min = df_pred_10min.dropna(subset=['时间'])
df_routes_info['线路编码'] = df_routes_info['线路编码'].str.strip(); df_fleet_info['车队编码'] =
df_fleet_info['车队编码'].str.strip()
df_routes_info['发运节点'] = pd.to_datetime(df_routes_info['发运节点'],
format="%H:%M:%S").dt.time; df_pred_10min['线路编码'] = df_pred_10min['线路编码']

```



```

].str.strip()
df_routes_merged = pd.merge(df_routes_info, df_fleet_info, on=' 车 队 编 码 ', how='left');
route_details = df_routes_merged.set_index('线路编码').T.to_dict()
print("数据加载和预处理完成。")

```

```

# --- 步骤一 & 二 (合并): 生成潜在任务 & 最优串点 ---
print("生成潜在运输任务及最优串点...")
# ... (Set Covering 串点优化代码与上个版本相同, 最终生成 final_tasks) ...
potential_full_tasks = []; leftover_candidates = []
all_routes = df_pred_10min['线路编码'].unique()
for route_code in tqdm(all_routes, desc="处理线路生成任务"):
    route_info = route_details.get(route_code)
    if not route_info: continue
    node_time = route_info['发运节点']; cumulative_volume = 0; full_loads_dispatched = 0
    if node_time == time_0600_dt: node_time_dt = node_time_0600; window_start_dt =
pd.Timestamp.combine(PREDICTION_DATE.date() - timedelta(days=1), time_2100_dt);
window_end_dt = node_time_dt
    elif node_time == time_1400_dt: node_time_dt = node_time_1400; window_start_dt =
pd.Timestamp.combine(PREDICTION_DATE.date(), time_1100_dt); window_end_dt =
node_time_dt
    else: continue
    df_route_window = df_pred_10min[(df_pred_10min[' 线 路 编 码 ' ] == route_code) &
(df_pred_10min[' 时 间 ' ] >= window_start_dt) & (df_pred_10min[' 时 间 ' ] <
window_end_dt)].sort_values(by='时间')
    for index, row in df_route_window.iterrows():
        current_time_dt = row['时间']; cumulative_volume += row['包裹量']
        while cumulative_volume >= VEHICLE_CAPACITY_NORMAL:
            task_id = f"{route_code}_Full_{full_loads_dispatched + 1}"; earliest_dispatch =
current_time_dt; latest_dispatch = node_time_dt
            potential_full_tasks.append({'task_id': task_id, 'type': 'Full', 'routes': [route_code],
'volume': VEHICLE_CAPACITY_NORMAL, 'earliest_dispatch_dt': earliest_dispatch,
'latest_dispatch_dt': latest_dispatch, 'node_time_dt': node_time_dt, 'required_fleet': route_info['车
队编码'], 'travel_time_hr': route_info['在途时长'], 'variable_cost': route_info['自有变动成本']})
            cumulative_volume -= VEHICLE_CAPACITY_NORMAL; full_loads_dispatched
+= 1
        if cumulative_volume > 1e-6:
            leftover_candidates.append({'original_index': len(leftover_candidates), 'route_code':
route_code, 'volume': cumulative_volume, 'node_time_dt': node_time_dt, 'required_fleet':
route_info['车队编码'], 'travel_time_hr': route_info['在途时长'], 'variable_cost': route_info['自有
变动成本'], 'base_route': route_code.rsplit(' - ', 1)[0]})

print("执行最优串点 (CP-SAT Set Covering)...")
consolidated_tasks = []; unconsolidated_leftovers = []; grouped_leftovers = {}

```

```

for item in leftover_candidates: key = (item['base_route'].split(' - ')[0], item['node_time_dt']);
grouped_leftovers.setdefault(key, []).append(item)
MAX_GROUP_SIZE_FOR_OPTIMAL = 50
for key, items in tqdm(grouped_leftovers.items(), desc="处理串点组"):

    num_items = len(items)
    if num_items == 0: continue
    if num_items == 1: item = items[0]; task_id = f'{item["route_code"]}_Leftover';
unconsolidated_leftovers.append({'task_id': task_id, 'type': 'Leftover', 'routes': [item['route_code']],
'volume': item['volume'], 'earliest_dispatch_dt': item['node_time_dt'], 'latest_dispatch_dt':
item['node_time_dt'], 'node_time_dt': item['node_time_dt'], 'required_fleet': item['required_fleet'],
'travel_time_hr': item['travel_time_hr'], 'variable_cost': item['variable_cost']}); continue
    if num_items > MAX_GROUP_SIZE_FOR_OPTIMAL:
        for item in items: task_id = f'{item["route_code"]}_Leftover_LargeGroup';
unconsolidated_leftovers.append({'task_id': task_id, 'type': 'Leftover', 'routes': [item['route_code']],
'volume': item['volume'], 'earliest_dispatch_dt': item['node_time_dt'], 'latest_dispatch_dt':
item['node_time_dt'], 'node_time_dt': item['node_time_dt'], 'required_fleet': item['required_fleet'],
'travel_time_hr': item['travel_time_hr'], 'variable_cost': item['variable_cost']})
        continue
    model_sc = cp_model.CpModel(); item_indices = range(num_items); potential_sets = [];
set_counter = 0
    for combo_size in range(2, MAX_STOPS_CONSOLIDATION + 1):
        for combo_indices in combinations(item_indices, combo_size):
            combo_items = [items[i] for i in combo_indices]; total_volume =
sum(item['volume'] for item in combo_items)
            if total_volume <= VEHICLE_CAPACITY_NORMAL:
potential_sets.append((set_counter, list(combo_indices), total_volume)); set_counter += 1
            set_vars = [model_sc.NewBoolVar(f'set_{s[0]}') for s in potential_sets]; single_vars =
[model_sc.NewBoolVar(f'single_{i}') for i in item_indices]
            item_to_sets = {i: [] for i in item_indices}
            for s_idx, s_data in enumerate(potential_sets):
                for item_idx in s_data[1]: item_to_sets[item_idx].append(s_idx)
            for i in item_indices: model_sc.Add(sum(set_vars[s_idx] for s_idx in item_to_sets[i]) +
single_vars[i] == 1)
            model_sc.Minimize(sum(set_vars) + sum(single_vars))
            solver_sc = cp_model.CpSolver(); status_sc = solver_sc.Solve(model_sc)
            if status_sc == cp_model.OPTIMAL or status_sc == cp_model.FEASIBLE:
                for s_idx, s_data in enumerate(potential_sets):
                    if solver_sc.Value(set_vars[s_idx]) == 1:
                        combo_indices = s_data[1]; combo_items = [items[i] for i in combo_indices];
task_id = f'Consolidated_{key[0]}_{key[1].strftime("%H%M")}_{s_idx}'; routes =
[item['route_code'] for item in combo_items]; max_travel_time = max(item['travel_time_hr'] for
item in combo_items); max_var_cost = max(item['variable_cost'] for item in combo_items)
                        consolidated_tasks.append({'task_id': task_id, 'type': 'Consolidated', 'routes':

```

```

routes, 'volume': s_data[2], 'earliest_dispatch_dt': key[1], 'latest_dispatch_dt': key[1],
'node_time_dt': key[1], 'required_fleet': combo_items[0]['required_fleet'], 'travel_time_hr':
max_travel_time, 'variable_cost': max_var_cost})
    for i in item_indices:
        if solver_sc.Value(single_vars[i]) == 1:
            item = items[i]; task_id = f'{item["route_code"]}_Leftover';
unconsolidated_leftovers.append({'task_id': task_id, 'type': 'Leftover', 'routes': [item['route_code']],
'volume': item['volume'], 'earliest_dispatch_dt': item['node_time_dt'], 'latest_dispatch_dt':
item['node_time_dt'], 'node_time_dt': item['node_time_dt'], 'required_fleet': item['required_fleet'],
'travel_time_hr': item['travel_time_hr'], 'variable_cost': item['variable_cost']})
        else: # Fallback
            for item in items: task_id = f'{item["route_code"]}_Leftover_SolveFail';
unconsolidated_leftovers.append({'task_id': task_id, 'type': 'Leftover', 'routes': [item['route_code']],
'volume': item['volume'], 'earliest_dispatch_dt': item['node_time_dt'], 'latest_dispatch_dt':
item['node_time_dt'], 'node_time_dt': item['node_time_dt'], 'required_fleet': item['required_fleet'],
'travel_time_hr': item['travel_time_hr'], 'variable_cost': item['variable_cost']})

final_tasks = potential_full_tasks + consolidated_tasks + unconsolidated_leftovers
print(f"任务生成与最优串点完成，共 {len(final_tasks)} 个运输任务。")

# --- 步骤三：构建并求解扩展的 CP-SAT 调度模型（精确时长）---
print("构建并求解 CP-SAT 调度模型（精确时长与容器决策）...")
model_sched = cp_model.CpModel()
# --- 数据准备 for CP-SAT ---
num_tasks = len(final_tasks); task_indices = range(num_tasks)

own_vehicles = []; vehicle_id_counter = 0
for idx, row in df_fleet_info.iterrows():
    for i in range(row['自有车数量']): own_vehicles.append({'id': f'Own_{row['车队编码']}_{i+1}', 'fleet': row['车队编码'], 'is_external': False}); vehicle_id_counter += 1
num_own_vehicles = len(own_vehicles); external_vehicles = [{'id': 'External', 'fleet': 'External',
'is_external': True}]; all_vehicles = own_vehicles + external_vehicles; num_all_vehicles =
len(all_vehicles); vehicle_indices = range(num_all_vehicles)
horizon_end_min = time_to_minutes(OUTPUT_END_TIME + timedelta(hours=12)) # Ensure
horizon is long enough
task_data = []
for i, task in enumerate(final_tasks):
    earliest_start_min = time_to_minutes(task['earliest_dispatch_dt']); latest_start_min =
time_to_minutes(task['latest_dispatch_dt'])
    if latest_start_min < earliest_start_min: latest_start_min = earliest_start_min
    travel_time_min = int(task['travel_time_hr'] * 60)
    # 只计算基础在途时间，装卸时间将由容器决策决定
    task_data.append({'id': i, 'task_obj': task, 'earliest_start': earliest_start_min, 'latest_start':

```

```

latest_start_min,
        'travel_time': travel_time_min, # 存储基础在途时间
        'volume': task['volume'], 'required_fleet': task['required_fleet'],
        'variable_cost': task['variable_cost'], 'external_cost':
task['variable_cost'] * EXTERNAL_COST_MULTIPLIER}))

# --- 创建 CP-SAT 变量 ---
start_vars = [model_sched.NewIntVar(task_data[i]['earliest_start'], task_data[i]['latest_start'],
f'start_{i}') for i in task_indices]
assignment_vars = {} # (task_id, vehicle_id) -> BoolVar
for i in task_indices:
    for v_idx in vehicle_indices: assignment_vars[i, v_idx] =
model_sched.NewBoolVar(f'assign_{i}_{all_vehicles[v_idx]["id"]}')
vehicle_used_vars = [model_sched.NewBoolVar(f'used_{v["id"]}') for v in own_vehicles]
use_container_vars = [model_sched.NewBoolVar(f'use_container_{i}') for i in task_indices]

# <<< 修改/新增：创建精确的任务时长和结束时间变量 >>>
task_duration_vars = [] # 车辆被占用的时长 (装载+单程在途)
end_vars = [] # 车辆占用结束的时间点
completion_time_vars = [] # 车辆完成任务返回的时间 (如果需要用于其他约束)

for i in task_indices:
    travel_time_min = task_data[i]['travel_time']
    # 任务占用时长 = travel_time + loading_time
    # loading_time = 10 * use_container + 45 * (1 - use_container) = 45 - 35 * use_container
    duration_i = model_sched.NewIntVar(LOADING_TIME_CONTAINER_MIN +
travel_time_min, LOADING_TIME_NORMAL_MIN + travel_time_min, f'duration_{i}')
    model_sched.Add(duration_i == travel_time_min + LOADING_TIME_NORMAL_MIN -
(LOADING_TIME_NORMAL_MIN - LOADING_TIME_CONTAINER_MIN) *
use_container_vars[i])
    task_duration_vars.append(duration_i)

# <<< 新增：显式创建 end_vars[i] 并添加约束 >>>
end_i = model_sched.NewIntVar(0, horizon_end_min, f'end_{i}') # 创建结束时间变量
model_sched.Add(end_i == start_vars[i] + duration_i) # 添加约束 end = start + duration
end_vars.append(end_i) # 加入列表

# 完整周转时间 = travel*2 + loading + unloading
# unloading_time = 10 * use_container + 45 * (1 - use_container) = 45 - 35 * use_container
# round_trip = travel*2 + (45 - 35*use_cont) + (45 - 35*use_cont) = travel*2 + 90 -
70*use_cont
round_trip_i = model_sched.NewIntVar(
    travel_time_min * 2 + LOADING_TIME_CONTAINER_MIN +

```

```

UNLOADING_TIME_CONTAINER_MIN,
    travel_time_min * 2 + LOADING_TIME_NORMAL_MIN +
UNLOADING_TIME_NORMAL_MIN,
    f'round_trip_{i}'
)
model_sched.Add(round_trip_i == travel_time_min * 2 +
(LOADING_TIME_NORMAL_MIN + UNLOADING_TIME_NORMAL_MIN) - \
    (LOADING_TIME_NORMAL_MIN
LOADING_TIME_CONTAINER_MIN + UNLOADING_TIME_NORMAL_MIN -
UNLOADING_TIME_CONTAINER_MIN) * use_container_vars[i])

# 车辆完成返回时间 = start + round_trip
completion_i = model_sched.NewIntVar(0, horizon_end_min, f'completion_{i}')
model_sched.Add(completion_i == start_vars[i] + round_trip_i)
completion_time_vars.append(completion_i)

# --- 添加 CP-SAT 约束 ---
print("添加调度约束 (精确时长)...")
# 1. 任务分配约束 (不变)
for i in task_indices: model_sched.Add(sum(assignment_vars[i, v_idx] for v_idx in
vehicle_indices) == 1) # 所有任务都执行

# <<<车辆无重叠约束 (恢复使用 end_vars[i] 作为 end 参数)>>>

for v_idx, vehicle in enumerate(own_vehicles):
    intervals_for_vehicle = []
    for i in task_indices:

        interval = model_sched.NewOptionalIntervalVar(
            start = start_vars[i],
            size = task_duration_vars[i],
            end = end_vars[i],
            is_present = assignment_vars[i, v_idx],
            name = f'interval_{i}_{vehicle["id"]}'
        )
        intervals_for_vehicle.append(interval)
    model_sched.AddNoOverlap(intervals_for_vehicle)

# 3. 车队归属约束
for i in task_indices:
    required_fleet = task_data[i]['required_fleet']
    for v_idx, vehicle in enumerate(all_vehicles):

```

```

        if not vehicle['is_external'] and vehicle['fleet'] != required_fleet:
model_sched.Add(assignment_vars[i, v_idx] == 0)
# 4. 车辆使用变量约束
for v_idx, vehicle in enumerate(own_vehicles):
    model_sched.Add(sum(assignment_vars[i, v_idx] for i in task_indices) >
0).OnlyEnforceIf(vehicle_used_vars[v_idx])
    model_sched.Add(sum(assignment_vars[i, v_idx] for i in task_indices) ==
0).OnlyEnforceIf(vehicle_used_vars[v_idx].Not())

# 5. 容器使用与容量约束
for i in task_indices:
    model_sched.Add(task_data[i]['volume'] <=
VEHICLE_CAPACITY_CONTAINER).OnlyEnforceIf(use_container_vars[i])
# 6. 外部车辆不使用容器约束
external_vehicle_index = num_own_vehicles
for i in task_indices: model_sched.Add(use_container_vars[i] ==
0).OnlyEnforceIf(assignment_vars[i, external_vehicle_index])
# 7. 自有车才能用容器的约束 (使用线性化)
is_assigned_to_own_vehicle = [model_sched.NewBoolVar(f'is_assigned_own_{i}') for i in
task_indices]
for i in task_indices:
    sum_assigned_own = sum(assignment_vars[i, v_idx] for v_idx in range(num_own_vehicles))
    model_sched.Add(sum_assigned_own == 1).OnlyEnforceIf(is_assigned_to_own_vehicle[i])
    model_sched.Add(sum_assigned_own ==
0).OnlyEnforceIf(is_assigned_to_own_vehicle[i].Not())
    model_sched.AddImplication(use_container_vars[i], is_assigned_to_own_vehicle[i])

# --- 定义目标函数 ---
print("定义调度目标函数 (基于精确时长)...")
# 1. 成本项 (外部成本可加因子)
total_fixed_cost = sum(vehicle_used_vars[v_idx] * VEHICLE_FIXED_COST for v_idx in
range(num_own_vehicles))
total_variable_cost = sum(assignment_vars[i, v_idx] * int(task_data[i]['variable_cost']) for i in
task_indices for v_idx in range(num_own_vehicles))
total_external_cost = sum(assignment_vars[i, external_vehicle_index] *
int(task_data[i]['external_cost']) for i in task_indices) # 使用基础估算外部成本
total_cost = total_fixed_cost + total_variable_cost + total_external_cost

# 2. 周转率项
total_own_tasks = sum(assignment_vars[i, v_idx] for i in task_indices for v_idx in
range(num_own_vehicles))
M_turnover = num_tasks * 2; turnover_objective_term = M_turnover * num_own_vehicles -
total_own_tasks if num_own_vehicles > 0 else 0

```

3. 填充率项

```
unused_capacity_terms = []
for i in task_indices:
    task_capacity_var = model_sched.NewIntVar(VEHICLE_CAPACITY_CONTAINER,
VEHICLE_CAPACITY_NORMAL, f'capacity_{i}')
    model_sched.Add(task_capacity_var == VEHICLE_CAPACITY_NORMAL -
(VEHICLE_CAPACITY_NORMAL - VEHICLE_CAPACITY_CONTAINER) *
use_container_vars[i])
    unused_capacity_i = model_sched.NewIntVar(0, VEHICLE_CAPACITY_NORMAL,
f'unused_cap_{i}')
    model_sched.Add(unused_capacity_i == task_capacity_var - int(task_data[i]['volume'])) #
直接计算，因为所有任务都执行
    unused_capacity_terms.append(unused_capacity_i)
total_unused_capacity = sum(unused_capacity_terms)
fill_rate_objective_term = total_unused_capacity
```

组合目标函数 (使用权重平衡)

```
SCALE_FACTOR = 100
int_w_cost = int(W_COST * SCALE_FACTOR); int_w_turnover = int(W_TURNOVER *
SCALE_FACTOR); int_w_fillrate = int(W_FILLRATE * SCALE_FACTOR)
```

```
model_sched.Minimize(
    total_cost * int_w_cost
    + turnover_objective_term * int_w_turnover
    + fill_rate_objective_term * int_w_fillrate
)
```

--- 求解模型 ---

```
print("开始求解精确时长的 CP-SAT 调度模型...")
solver_sched = cp_model.CpSolver(); solver_sched.parameters.log_search_progress = True;
solver_sched.parameters.max_time_in_seconds = 300.0 # 可以给更长时间
status_sched = solver_sched.Solve(model_sched)
```

--- 处理结果 ---

```
print("处理调度求解结果...")
```

```
solution_found_sched = (status_sched == cp_model.OPTIMAL or status_sched ==
cp_model.FEASIBLE)
results_list_final_p3 = []
if solution_found_sched:
    print(f' 求解成功！ 状态：{solver_sched.StatusName(status_sched)}\n 最优目标值：
{solver_sched.ObjectiveValue()}')
```

```

assigned_tasks_count = 0
for i in task_indices:
    assigned_vehicle_id = None; container_used = False; dispatch_time_dt = None
    for v_idx in vehicle_indices:
        if solver_sched.Value(assignment_vars[i, v_idx]) == 1:
            assigned_vehicle_id = all_vehicles[v_idx]['id']; start_time_min =
solver_sched.Value(start_vars[i]); dispatch_time_dt = minutes_to_time(start_time_min)
            container_used = (solver_sched.Value(use_container_vars[i]) == 1)
            if all_vehicles[v_idx]['is_external']: container_used = False
            assigned_tasks_count += 1; break
    if assigned_vehicle_id and dispatch_time_dt:
        original_routes = ", ".join(task_data[i]['task_obj']['routes'])
        results_list_final_p3.append({'线路编码': original_routes, '日期':
PREDICTION_DATE.date(), '预计发运时间': dispatch_time_dt, '是否使用容器': "是" if
container_used else "否", '发运车辆': assigned_vehicle_id})
    print(f'共处理和分配了 {assigned_tasks_count} 个运输任务。")
    df_final_schedule_p3 = pd.DataFrame(results_list_final_p3); df_final_schedule_p3 =
df_final_schedule_p3[['线路编码','日期','预计发运时间','是否使用容器','发运车辆']]
    df_final_schedule_p3['日期'] = pd.to_datetime(df_final_schedule_p3['日期']);
df_final_schedule_p3['预计发运时间'] = pd.to_datetime(df_final_schedule_p3['预计发运时间'])
    try:
        print(f'更新/保存 {RESULT_TABLE4_PATH}...")
        df_final_schedule_p3['日期'] = df_final_schedule_p3['日期'].dt.strftime("%Y/%m/%d")
        df_final_schedule_p3['预计发运时间'] = df_final_schedule_p3['预计发运时间']
        ].dt.strftime("%H:%M:%S")
        df_final_schedule_p3.to_excel(RESULT_TABLE4_PATH, index=False,
engine='openpyxl')
        print(f'{RESULT_TABLE4_PATH} 更新/保存成功。")
    except Exception as e:
        print(f'更新/保存 {RESULT_TABLE4_PATH} 失败: {e}")
        backup_filename = os.path.join(OUTPUT_DIR,
"Result_Table4_Schedule_Backup.xlsx")
        pd.DataFrame(results_list_final_p3).to_excel(backup_filename, index=False)
        print(f'调度结果已备份至: {backup_filename}")
else:
    print("调度模型未能找到可行解或最优解。请检查模型约束或增加求解时间。")

```

--- 结束计时 ---

end_time = time.time(); total_time = end_time - start_time

print(f"\n 问题三(精确时长模型)处理完成！总耗时: {total_time:.2f} 秒")

11.4 第四题主代码程序：

```
import pandas as pd
```



```

import numpy as np
from tqdm import tqdm
import os
import warnings
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import time
from datetime import datetime, timedelta
# 引入 3D 绘图工具
from mpl_toolkits.mplot3d import Axes3D

warnings.filterwarnings('ignore')

# --- 设置 Matplotlib 字体 ---
FONT_NAME = 'Microsoft YaHei' # 尝试使用微软雅黑
try:
    plt.rcParams['font.sans-serif'] = [FONT_NAME]
    print(f'已尝试设置 Matplotlib 字体为: {FONT_NAME}')
except Exception as e:
    print(f'警告: 设置 Matplotlib 字体 '{FONT_NAME}' 失败: {e}。尝试备选 'SimHei'...")
    try:
        FONT_NAME = 'SimHei'
        plt.rcParams['font.sans-serif'] = [FONT_NAME]
        print(f'已尝试设置 Matplotlib 字体为: {FONT_NAME}')
    except Exception as e2:
        print(f'警告: 设置 Matplotlib 备选字体 '{FONT_NAME}' 也失败: {e2}。中文可能无法正常显示。")
plt.rcParams['axes.unicode_minus'] = False

# --- 0. 参数与常量 ---

PREDICTION_DATE = pd.to_datetime("2024-12-16"); TIME_HORIZON_START_DT =
pd.to_datetime("2024-12-15 00:00:00")
RESULT_TABLE2_PATH = '结果表 2.xlsx'; RESULT_TABLE4_PATH = '结果表 4.xlsx';
FLEET_INFO_PATH = '附件 5.xlsx'; ROUTE_INFO_PATH = '附件 1.xlsx'
OUTPUT_DIR = "./results_problem4"
VEHICLE_CAPACITY_NORMAL = 1000; VEHICLE_CAPACITY_CONTAINER = 800;
LOADING_TIME_NORMAL_MIN = 45; UNLOADING_TIME_NORMAL_MIN = 45
LOADING_TIME_CONTAINER_MIN = 10; UNLOADING_TIME_CONTAINER_MIN = 10;
VEHICLE_FIXED_COST = 100; EXTERNAL_COST_MULTIPLIER = 2.0
VOLUME_DEVIATION_LEVELS = [-0.3, -0.2, -0.1, 0.0, 0.1, 0.2, 0.3];
TIME_SHIFT_LEVELS_MIN = [-90, -60, -30, 0, 30, 60, 90]

```

--- 1. 加载所需数据 ---

```
start_time = time.time(); print("加载数据...")
try:
    df_pred_original = pd.read_excel(RESULT_TABLE2_PATH); df_schedule =
pd.read_excel(RESULT_TABLE4_PATH); df_routes_info =
pd.read_excel(ROUTE_INFO_PATH); df_fleet_info = pd.read_excel(FLEET_INFO_PATH)
except FileNotFoundError as e: print(f"错误: 找不到所需文件 - {e}。"); exit()
os.makedirs(OUTPUT_DIR, exist_ok=True); print("数据加载完成。")
```

--- 2. 数据预处理 ---

```
print("数据预处理...");
df_pred_original['日期'] = pd.to_datetime(df_pred_original['日期'])
try: df_pred_original['分钟起始'] = pd.to_datetime(df_pred_original['分钟起始'],
format='%H:%M:%S').dt.time
except: df_pred_original['分钟起始'] = pd.to_datetime(df_pred_original['分钟起始'].astype(str),
errors='coerce').dt.time
df_pred_original['包裹量'] = pd.to_numeric(df_pred_original['包裹量'], errors='coerce').fillna(0)
df_pred_original['时间'] = df_pred_original.apply(lambda row: pd.Timestamp.combine(row['日期']
.date(), row['分钟起始'])) if pd.notna(row['日期']) and pd.notna(row['分钟起始']) else pd.NaT,
axis=1)
df_pred_original = df_pred_original.dropna(subset=['时间']); df_pred_original['线路编码'] =
df_pred_original['线路编码'].str.strip(); df_pred_original.set_index('时间', inplace=True)
df_schedule['线路编码'] = df_schedule['线路编码'].str.strip(); df_schedule['日期'] =
pd.to_datetime(df_schedule['日期'])
try: df_schedule['预计发运时间'] = pd.to_datetime(df_schedule['预计发运时间'],
format='%H:%M:%S').dt.time
except: df_schedule['预计发运时间'] = pd.to_datetime(df_schedule['预计发运时间'].astype(str),
errors='coerce').dt.time
df_schedule['发运时刻'] = df_schedule.apply(lambda row: pd.Timestamp.combine(row['日期']
.date(), row['预计发运时间'])) if pd.notna(row['日期']) and pd.notna(row['预计发运时间']) else
pd.NaT, axis=1)
df_schedule = df_schedule.dropna(subset=['发运时刻']).sort_values(by='发运时刻')
df_routes_info['线路编码'] = df_routes_info['线路编码'].str.strip(); df_fleet_info['车队编码'] =
df_fleet_info['车队编码'].str.strip()
df_routes_merged = pd.merge(df_routes_info, df_fleet_info, on='车队编码', how='left');
route_details = df_routes_merged.set_index('线路编码').T.to_dict()
total_own_vehicles = df_fleet_info['自有车数量'].sum(); print("数据预处理完成。")
```

--- 3. 场景生成函数 ---

```
def generate_volume_scenario(df_original, deviation_ratio): df_scenario = df_original.copy();
df_scenario['包裹量'] = (df_scenario['包裹量'] * (1 + deviation_ratio)).round().clip(lower=0);
```

```

return df_scenario
def generate_time_shift_scenario(df_original, shift_minutes): df_scenario = df_original.copy();
time_delta = timedelta(minutes=shift_minutes); df_scenario.index = df_scenario.index +
time_delta; return df_scenario

```

--- 4. 仿真执行与 KPI 计算函数 ---

```

def run_simulation(schedule, demand_scenario, route_details_dict):
    actual_kpis = {}; total_actual_loaded_volume = 0; total_leftover_volume = 0;
    actual_available_volume = {route: 0.0 for route in demand_scenario['线路编码'].unique()}
    total_actual_demand = demand_scenario.groupby('线路编码')['包裹量'].sum();
    scheduled_own_tasks_count = 0; successful_own_tasks_count = 0; total_scheduled_trips =
    len(schedule); total_actual_cost = 0
    last_sim_time = TIME_HORIZON_START_DT
    for index, planned_dispatch in schedule.iterrows():
        dispatch_time = planned_dispatch['发运时刻']; vehicle_type = planned_dispatch['发运
        车辆']; use_container = (planned_dispatch['是否使用容器'] == '是'); is_external = vehicle_type
        == 'External'
        assigned_routes_str = planned_dispatch['线路编码']; assigned_routes = [r.strip() for r in
        assigned_routes_str.split(',')]
        time_increment = dispatch_time - last_sim_time
        if time_increment > timedelta(0):
            demand_increment = demand_scenario[(demand_scenario.index > last_sim_time)
            & (demand_scenario.index <= dispatch_time)]
            for route, volume in demand_increment.groupby('线路编码')['包裹量']
            .sum().items():
                if route in actual_available_volume: actual_available_volume[route] +=
                volume
            last_sim_time = dispatch_time
            planned_total_volume_task = 0; actual_loaded_task = 0; vehicle_capacity =
            VEHICLE_CAPACITY_CONTAINER if use_container and not is_external else
            VEHICLE_CAPACITY_NORMAL
            is_own_vehicle = not is_external
            if is_own_vehicle: scheduled_own_tasks_count += 1
            planned_volume_this_dispatch = vehicle_capacity # Simplified assumption
            current_task_leftover = 0; temp_loaded_task = 0
            for route in assigned_routes:
                plan_vol_route = planned_volume_this_dispatch / len(assigned_routes);
                avail_vol_route = actual_available_volume.get(route, 0); can_load_route = min(plan_vol_route,
                avail_vol_route)
                remaining_capacity = vehicle_capacity - temp_loaded_task; actual_load_route =
                min(can_load_route, remaining_capacity)
                temp_loaded_task += actual_load_route; actual_available_volume[route] -=
                actual_load_route

```

```

        actual_loaded_task    =    temp_loaded_task;    total_actual_loaded_volume    +=
actual_loaded_task
        if is_own_vehicle and actual_loaded_task > 0: successful_own_tasks_count += 1
        route_info = route_details_dict.get(assigned_routes[0])
        if route_info:
            cost_val = route_info.get('自有变动成本', 0);
            if pd.isna(cost_val): cost_val = 0
            if is_external:    total_actual_cost    +=    cost_val    *
EXTERNAL_COST_MULTIPLIER
            else: total_actual_cost += cost_val
        own_vehicles_used = schedule[~schedule['发运车辆'].str.startswith('External')]['发运车辆
'].nunique()
        total_actual_cost += own_vehicles_used * VEHICLE_FIXED_COST
        total_actual_demand_sum = total_actual_demand.sum()
        actual_kpis['Total Cost'] = total_actual_cost; actual_kpis['Own Vehicle Turnover Rate'] =
successful_own_tasks_count / total_own_vehicles if total_own_vehicles > 0 else 0
        actual_kpis['Avg Parcels per Vehicle'] = total_actual_loaded_volume / total_scheduled_trips
        if total_scheduled_trips > 0 else 0
        actual_kpis['On-Time Dispatch Rate'] = total_actual_loaded_volume /
total_actual_demand_sum if total_actual_demand_sum > 0 else 1.0
        final_leftover_volume = sum(v for v in actual_available_volume.values() if v > 1e-6)
        actual_kpis['Total Leftover Volume'] = final_leftover_volume
        return actual_kpis

```

--- 5. 运行仿真与收集结果 ---

```

print("运行基准场景仿真..."); baseline_kpis = run_simulation(df_schedule, df_pred_original,
route_details); print("基准 KPIs:", baseline_kpis)
results_all_scenarios = []; results_all_scenarios.append({'Scenario': 'Baseline (Ideal)', 'Deviation
Type': 'None', 'Level': 0, **baseline_kpis})
print("运行总量偏差场景仿真...");
for dev in tqdm(VOLUME_DEVIATION_LEVELS, desc="总量偏差场景"):
    if abs(dev) < 1e-6: continue
    scenario_demand = generate_volume_scenario(df_pred_original, dev); scenario_kpis =
run_simulation(df_schedule, scenario_demand, route_details);
results_all_scenarios.append({'Scenario': f'Volume {dev*100:+.0f}%', 'Deviation Type': 'Volume',
'Level': dev, **scenario_kpis})
print("运行时间漂移场景仿真...");
for shift in tqdm(TIME_SHIFT_LEVELS_MIN, desc="时间漂移场景"):
    if abs(shift) < 1e-6: continue
    scenario_demand = generate_time_shift_scenario(df_pred_original, shift); scenario_kpis =
run_simulation(df_schedule, scenario_demand, route_details);
results_all_scenarios.append({'Scenario': f'Time Shift {shift:+}min', 'Deviation Type': 'Time Shift',
'Level': shift, **scenario_kpis})

```

```

df_results = pd.DataFrame(results_all_scenarios); print("所有场景仿真完成。")
results_filename = os.path.join(OUTPUT_DIR, "sensitivity_analysis_results.xlsx");
df_results.to_excel(results_filename, index=False); print(f"详细仿真结果已保存至：
{results_filename}")

# --- 6. 可视化分析结果 (全面增强+修复标题+新增 3D) ---
print("生成增强版可视化图表 (修复标题, 新增 3D)...")

# --- 全局样式和参数 ---
sns.set_theme(style="whitegrid", palette="muted", font=FONT_NAME) # 显式设置字体
plt.rcParams['figure.dpi'] = 300; plt.rcParams['savefig.dpi'] = 300
TITLE_FONTSIZE = 16; SUBPLOT_TITLE_FONTSIZE = 14; LABEL_FONTSIZE = 12;
TICK_FONTSIZE = 10; ANNOT_FONTSIZE = 9; LEGEND_FONTSIZE = 10

kpi_names = ['Total Cost', 'Own Vehicle Turnover Rate', 'Avg Parcels per Vehicle',
             'On-Time Dispatch Rate', 'Total Leftover Volume']
kpi_units = {'Total Cost': '元', 'Own Vehicle Turnover Rate': '次/车', 'Avg Parcels per Vehicle': '件/
车',
             'On-Time Dispatch Rate': '%', 'Total Leftover Volume': '件'}
baseline_values = df_results[df_results['Scenario'] == 'Baseline (Ideal)'].iloc[0]

# --- 图表 6.1: KPI 对总量偏差的敏感性 (折线图) ---
df_vol = df_results[df_results['Deviation Type'].isin(['Volume', 'None'])].sort_values('Level')
fig1, axes1 = plt.subplots(2, 3, figsize=(18, 10)) # 创建 figure 和 axes 对象
fig1.suptitle("KPI 对预测总量偏差的敏感性 (相对基准变化率)", fontsize=TITLE_FONTSIZE,
y=1.0) # 调整 y 确保可见
axes1 = axes1.flatten() # 将 2x3 的 axes 数组展平
for i, kpi in enumerate(kpi_names):
    ax = axes1[i]; base_val = baseline_values[kpi]
    if abs(base_val) > 1e-9: relative_change = ((df_vol[kpi] / base_val) - 1) * 100
    else: relative_change = df_vol[kpi]
    sns.lineplot(x=df_vol['Level'] * 100, y=relative_change, marker='o', linewidth=2, ax=ax)
    ax.set_title(f'{kpi} 变化率 ({kpi_units.get(kpi, "")})',
    fontsize=SUBPLOT_TITLE_FONTSIZE) # 子图标题加入单位
    ax.set_xlabel("预测总量偏差 (%)", fontsize=LABEL_FONTSIZE); ax.set_ylabel(f'相对基
准变化 (%)' if abs(base_val) > 1e-9 else '绝对值', fontsize=LABEL_FONTSIZE)
    ax.axhline(0, color='grey', linestyle='--', lw=1); ax.tick_params(axis='both', which='major',
    labelsize=TICK_FONTSIZE)
# 隐藏最后一个空的子图 (如果是 2x3)
if len(kpi_names) < len(axes1): fig1.delaxes(axes1[-1])
plt.tight_layout(rect=[0, 0.03, 1, 0.97]) # 调整矩形区域
plt.savefig(os.path.join(OUTPUT_DIR, "vis_1_kpi_sensitivity_volume.png"), dpi=300);
plt.close(fig1)

```

```
print(" - 图表 6.1: 总量偏差敏感性图已生成。")
```

```
# --- 图表 6.2: KPI 对时间漂移的敏感性 (折线图) ---
```

```
df_time = df_results[df_results['Deviation Type'].isin(['Time Shift', 'None'])].sort_values('Level')
fig2, axes2 = plt.subplots(2, 3, figsize=(18, 10)); fig2.suptitle("KPI 对预测时间漂移的敏感性
(相对基准变化率)", fontsize=TITLE_FONT_SIZE, y=1.0)
axes2 = axes2.flatten()
for i, kpi in enumerate(kpi_names):
    ax = axes2[i]; base_val = baseline_values[kpi]
    if abs(base_val) > 1e-9: relative_change = ((df_time[kpi] / base_val) - 1) * 100
    else: relative_change = df_time[kpi]
    sns.lineplot(x=df_time['Level'], y=relative_change, marker='o', linewidth=2, ax=ax)
    ax.set_title(f'{kpi} 变化率 ({kpi_units.get(kpi, "")})',
    fontsize=SUBPLOT_TITLE_FONT_SIZE); ax.set_xlabel("预测时间漂移 (分钟)",
    fontsize=LABEL_FONT_SIZE); ax.set_ylabel(f'相对基准变化 (%)' if abs(base_val) > 1e-9 else '
    绝对值', fontsize=LABEL_FONT_SIZE)
    ax.axhline(0, color='grey', linestyle='--', lw=1); ax.tick_params(axis='both', which='major',
    labelsize=TICK_FONT_SIZE)
if len(kpi_names) < len(axes2): fig2.delaxes(axes2[-1])
plt.tight_layout(rect=[0, 0.03, 1, 0.97]); plt.savefig(os.path.join(OUTPUT_DIR,
"vis_2_kpi_sensitivity_time_shift.png"), dpi=300); plt.close(fig2)
print(" - 图表 6.2: 时间漂移敏感性图已生成。")
```

```
# --- 图表 6.3: 关键场景 KPI 对比 (柱状图, 2x2 布局) ---
```

```
scenarios_to_compare = ['Baseline (Ideal)', 'Volume -20%', 'Volume +20%', 'Time Shift -60min',
'Time Shift +60min']
df_compare = df_results[df_results['Scenario'].isin(scenarios_to_compare)].set_index('Scenario')
kpis_to_compare = ['Total Cost', 'Own Vehicle Turnover Rate', 'Avg Parcels per Vehicle', 'Total
Leftover Volume']
fig3, axes3 = plt.subplots(2, 2, figsize=(14, 10)); fig3.suptitle("关键场景下核心 KPI 对比",
    fontsize=TITLE_FONT_SIZE, y=1.0)
axes3 = axes3.flatten()
for i, kpi in enumerate(kpis_to_compare):
    ax = axes3[i]; data_to_plot = df_compare[kpi]; bars = sns.barplot(x=data_to_plot.index,
    y=data_to_plot, ax=ax, palette="Blues_d")
    ax.set_title(f'{kpi}\n({kpi_units.get(kpi, "")})', fontsize=SUBPLOT_TITLE_FONT_SIZE);
    ax.set_xlabel("模拟场景", fontsize=LABEL_FONT_SIZE); ax.set_ylabel("KPI 值",
    fontsize=LABEL_FONT_SIZE)
    ax.tick_params(axis='x', rotation=10, labelsize=TICK_FONT_SIZE); ax.tick_params(axis='y',
    labelsize=TICK_FONT_SIZE)
    for bar in bars.patches: bars.annotate(format(bar.get_height(), '.1f'), (bar.get_x() +
    bar.get_width() / 2., bar.get_height()), ha = 'center', va = 'bottom', size=ANNOT_FONT_SIZE,
```

```
xytext = (0, 5), textcoords = 'offset points')
plt.tight_layout(rect=[0, 0.03, 1, 0.96]); plt.savefig(os.path.join(OUTPUT_DIR,
"vis_3_kpi_comparison_key_scenarios_2x2.png"), dpi=300); plt.close(fig3)
print(" - 图表 6.3: 关键场景 KPI 对比图 (2x2) 已生成。")
```

--- 图表 6.4: KPI 分布的小提琴图 (组合展示) ---

```
df_plot_dist = df_results[df_results['Level'] != 0]
fig4, axes4 = plt.subplots(2, 3, figsize=(18, 12)); fig4.suptitle("不同偏差类型下 KPI 分布（小提琴图 + 散点图）", fontsize=TITLE_FONT_SIZE, y=1.0)
axes4 = axes4.flatten()
for i, kpi in enumerate(kpi_names):
    ax = axes4[i]; sns.violinplot(data=df_plot_dist, x='Deviation Type', y=kpi, inner='quartile',
palette='muted', cut=0, ax=ax)
    sns.stripplot(data=df_plot_dist, x='Deviation Type', y=kpi, color=".3", size=3, alpha=0.6,
ax=ax)
    ax.set_title(f'{kpi} 分布', fontsize=SUBPLOT_TITLE_FONT_SIZE); ax.set_xlabel("偏差类型",
    fontsize=LABEL_FONT_SIZE); ax.set_ylabel(f'KPI 值 ({kpi_units.get(kpi, "")})',
    fontsize=LABEL_FONT_SIZE)
    ax.axhline(baseline_values[kpi], color='red', linestyle=':', lw=1.5, label='Baseline (Ideal)')
    ax.tick_params(axis='both', which='major', labelsize=TICK_FONT_SIZE);
ax.legend(fontsize=LEGEND_FONT_SIZE)
if len(kpi_names) < len(axes4): fig4.delaxes(axes4[-1])
plt.tight_layout(rect=[0, 0.03, 1, 0.97]); plt.savefig(os.path.join(OUTPUT_DIR,
"vis_4_kpi_distribution_violin.png"), dpi=300); plt.close(fig4)
print(" - 图表 6.4: KPI 分布小提琴图已生成。")
```

--- 图表 6.5: KPI 相关性热力图 ---

... (与上一版本类似) ...

```
kpi_numeric_cols = [kpi for kpi in kpi_names if pd.api.types.is_numeric_dtype(df_results[kpi])]
df_kpis_only = df_results[kpi_numeric_cols]; correlation_matrix = df_kpis_only.corr()
plt.figure(figsize=(8, 7)); # 调整大小适应旋转标签
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5,
annot_kws={"size": 10})
plt.title('KPI 指标之间的相关性热力图', fontsize=TITLE_FONT_SIZE); plt.xticks(rotation=30,
ha='right', fontsize=TICK_FONT_SIZE); plt.yticks(rotation=0, fontsize=TICK_FONT_SIZE)
plt.tight_layout(); plt.savefig(os.path.join(OUTPUT_DIR, "vis_5_kpi_correlation_heatmap.png"),
dpi=300); plt.close()
print(" - 图表 6.5: KPI 相关性热力图已生成。")
```

--- 图表 6.6: 关键场景雷达图 ---

... (与上一版本类似, 确保标题和标签清晰) ...

```
def make_radar_chart(data, categories, title, filename):
    N = len(categories); angles = [n / float(N) * 2 * np.pi for n in range(N)]; angles += angles[:1]
```

```

fig, ax = plt.subplots(figsize=(8, 8), subplot_kw=dict(polar=True))
plt.xticks(angles[:-1], categories, color='grey', size=10) # 减小标签字号防止重叠
ax.set_rlabel_position(30) # 调整半径标签位置
plt.yticks(np.arange(0, 1.1, 0.2), ["0", "0.2", "0.4", "0.6", "0.8", "1.0"], color="grey", size=9);
plt.ylim(0, 1.1)
colors = plt.cm.get_cmap("Set1", len(data))
for i, (name, row) in enumerate(data.iterrows()):
    values = row.values.flatten().tolist(); values += values[:1]
    ax.plot(angles, values, color=colors(i), linewidth=2, linestyle='solid', label=name)
    ax.fill(angles, values, color=colors(i), alpha=0.2)
plt.title(title, size=TITLE_FONTSIZE, y=1.12)
ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1), fontsize=LEGEND_FONTSIZE) #
调整图例位置
plt.savefig(filename, dpi=300, bbox_inches='tight'); plt.close(fig)
radar_scenarios = ['Baseline (Ideal)', 'Volume -30%', 'Volume +30%', 'Time Shift -90min', 'Time
Shift +90min']
radar_kpis = ['Total Cost', 'Own Vehicle Turnover Rate', 'Avg Parcels per Vehicle', 'On-Time
Dispatch Rate', 'Total Leftover Volume']
df_radar_raw =
df_results[df_results['Scenario'].isin(radar_scenarios)].set_index('Scenario')[radar_kpis]
df_radar_norm = pd.DataFrame(index=df_radar_raw.index)
for kpi in radar_kpis:
    vals = df_radar_raw[kpi]; min_val, max_val = vals.min(), vals.max(); kpi_label =
f'{kpi}\n({kpi_units.get(kpi,)})'
    if kpi in ['Total Cost', 'Total Leftover Volume']: kpi_label += "\n(反向,越高越好)"
    if max_val - min_val > 1e-6:
        if kpi in ['Total Cost', 'Total Leftover Volume']: df_radar_norm[kpi_label] = (max_val -
vals) / (max_val - min_val)
        else: df_radar_norm[kpi_label] = (vals - min_val) / (max_val - min_val)
    else: df_radar_norm[kpi_label] = 0.5
df_radar_norm.fillna(0.5, inplace=True); radar_categories = list(df_radar_norm.columns)
make_radar_chart(df_radar_norm, radar_categories, '关键场景 KPI 表现对比 (雷达图, 0-1 标准
化)', os.path.join(OUTPUT_DIR, "vis_6_kpi_radar_chart.png"))
print(" - 图表 6.6: 关键场景雷达图已生成。")

```

--- 图表 6.7: Pair Plot (KPIs 关系可视化) ---

```

print(" - 生成 Pair Plot (可能需要几秒钟)...")
pairplot_kpis = kpi_names # 选择要展示的 KPI
df_pair = df_results[df_results['Deviation Type'] != 'None'] # 排除基准点, 观察偏差影响
pair_plot = sns.pairplot(df_pair, vars=pairplot_kpis, hue='Deviation Type', plot_kws={'alpha': 0.6,
's': 40}, diag_kind='kde') # s 调整点大小, diag_kind='kde'用核密度估计
pair_plot.fig.suptitle('KPI 指标间的关系与分布 (按偏差类型着色)', y=1.01,
fontsize=TITLE_FONTSIZE)
pair_plot.fig.tight_layout(rect=[0, 0.03, 1, 0.98])

```



```

pair_plot.savefig(os.path.join(OUTPUT_DIR, "vis_7_kpi_pairplot.png"), dpi=300)
plt.close()
print(" - 图表 6.7: Pair Plot 已生成。")

# --- 图表 6.8: 3D 散点图示例 (总量偏差 vs 成本 vs 滞留量) ---
print(" - 生成 3D 散点图...")
df_vol_data_3d = df_results[df_results['Deviation Type'].isin(['Volume', 'None'])].copy()
df_vol_data_3d['Volume Deviation (%)'] = df_vol_data_3d['Level'] * 100

fig_3d = plt.figure(figsize=(10, 8))
ax_3d = fig_3d.add_subplot(111, projection='3d')

# 散点图
scatter = ax_3d.scatter(df_vol_data_3d['Volume Deviation (%)'],
                        df_vol_data_3d['Total Cost'],
                        df_vol_data_3d['Total Leftover Volume'],
                        c=df_vol_data_3d['Total Leftover Volume'], # 用滞留量的大小来
                        # 决定颜色
                        cmap='viridis', # 选择色谱
                        s=50, alpha=0.8) # s 调整点大小

# 添加颜色条
cbar = fig_3d.colorbar(scatter)
cbar.set_label('总滞留量 (件)', fontsize=LABEL_FONTSIZE)
cbar.ax.tick_params(labelsize=TICK_FONTSIZE)

# 设置轴标签和标题
ax_3d.set_xlabel('预测总量偏差 (%)', fontsize=LABEL_FONTSIZE, labelpad=10)
ax_3d.set_ylabel('总成本 (元)', fontsize=LABEL_FONTSIZE, labelpad=10)
ax_3d.set_zlabel('总滞留量 (件)', fontsize=LABEL_FONTSIZE, labelpad=10)
ax_3d.set_title('总量偏差对成本与滞留量的影响 (3D 散点图)', fontsize=TITLE_FONTSIZE,
y=1.05)

# 调整视角 (可选)
# ax_3d.view_init(elev=20., azim=-65)

# 调整刻度标签大小
ax_3d.tick_params(axis='both', which='major', labelsize=TICK_FONTSIZE)

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(os.path.join(OUTPUT_DIR, "vis_8_3d_scatter_volume_cost_leftover.png"), dpi=300)
plt.close(fig_3d)
print(" - 图表 6.8: 3D 散点图已生成。")

```

```
print("所有可视化图表生成完毕。")

# --- 结束计时 ---
end_time = time.time(); total_time = end_time - start_time
print(f"\n 问题四处理完成！总耗时: {total_time:.2f} 秒")
```